

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по теме:
**РАЗРАБОТКА СИСТЕМЫ ДЛЯ ПРОКСИРОВАНИЯ И
МОНИТОРИНГА СЕРВИСОВ-ЗАГЛУШЕК В СРЕДЕ
НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Студент: _____ / Медведев Клим Николаевич, 221-329 /
подпись *ФИО, группа*

Руководитель ВКР: _____ / Кружалов Алексей Сергеевич /
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Тема ВКР	
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	
Основные функции	1. Список функций
Используемые технологии и платформы	Перечисление технологий.

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Список задач
Состав технической документации	1. Список документации
Состав графической части	1. Список графической части

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«___»_____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«___»_____2026, _____ / Кружалов Алексей Сергеевич /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«___»_____2026, _____ / Медведев Клим Николаевич, 221-329 /
подпись *ФИО, группа*

АННОТАЦИЯ

Выпускная квалификационная работа (далее ВКР) на тему: «Тема».

Цель ВКР – ...ОИИАОИ

Наполнение аннотации смотрите в методичке.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	9
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	12
1.1 Исследование инфраструктуры нагрузочного тестирования и роли имитационных сервисов	12
1.2 Анализ существующих подходов к управлению сервисами-заглушками и проблема импортозамещения	13
1.3 Сравнительный анализ инструментальных средств разработки и обоснование технологического стека	15
1.4 Обзор аналогичных решений и обоснование разработки собственного программного комплекса	21
1.5 Формирование функциональных и технических требований к системе	28
1.6 Выводы по первой главе	32
Выводы по первой главе	32
2 ПРОЕКТИРОВАНИЕ И ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ УПРАВЛЕНИЯ	34
2.1 Разработка архитектуры распределённого программного комплекса	34
2.2 Проектирование структур данных для хранения конфигураций и состояний в Redis	49
2.3 Разработка макетов приложения	59
2.4 Реализация асинхронных механизмов управления процессами и ввода пользовательских параметров.	67
2.5 Разработка модуля динамического проксирования запросов на базе FastAPI и алгоритма Rate Limiting	76
2.6 Проектирование и программная реализация пользовательского интерфейса (Dashboard, управление кластером).	86
2.7 Реализация механизма экспорта метрик мониторинга	86

2.8	Выводы по второй главе.	86
3	ВНЕДРЕНИЕ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ РАЗРАБОТАННОГО ПРОГРАММНОГО КОМПЛЕКСА	87
3.1	Описание процесса развертывания системы в среде ОС Astra Linux (настройка Uvicorn/Gunicorn и службы Redis).	87
3.2	Тестирование механизмов автоматического восстановления состояний (персистентность Redis AOF) и перезапуска сервисов.	87
3.3	Экспериментальная оценка производительности прокси-модуля и эффективности алгоритма Rate Limiting под нагрузкой.	87
3.4	Анализ работы системы в кластерном режиме и оценка алгоритмов распределения нагрузки между узлами.	87
3.5	Рекомендации по дальнейшему развитию системы (шаблонизация конфигураций, Self-healing).	87
3.6	Выводы по третьей главе.	87
	ЗАКЛЮЧЕНИЕ	88
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	89
	ПРИЛОЖЕНИЕ А	92

ВВЕДЕНИЕ

Актуальность темы. В современной практике обеспечения качества сложных программных комплексов проведение нагрузочного тестирования требует создания стабильной и управляемой среды. Одной из ключевых проблем при организации такой среды является управление инфраструктурой имитационных сервисов (заглушек) – специализированного программного обеспечения, воспроизводящего поведение реальных внешних компонентов системы.

В условиях реализации политики импортозамещения в Российской Федерации актуальность приобретает разработка отечественных инструментов управления инфраструктурой, способных заменить зарубежные программные решения и обеспечить совместимость с сертифицированными отечественными операционными системами.

Цель работы – проектирование и разработка автоматизированной информационной системы для централизованного управления и мониторинга состояния распределённой инфраструктуры имитационных сервисов.

Задачи исследования:

- а) Провести анализ предметной области и существующих подходов к управлению имитационными средами в процессах обеспечения качества ПО.
- б) Спроектировать архитектуру программного комплекса, обеспечивающую гибкое масштабирование и работу в различных режимах.
- в) Обосновать выбор технологического стека и программных средств реализации системы.
- г) Разработать алгоритмы динамического проксирования трафика, механизмы контроля интенсивности запросов и программные интерфейсы для автоматизации управления.
- д) Реализовать систему хранения конфигураций и контроля состояний исполняемых файлов.

е) Провести тестирование разработанного комплекса в среде ОС Astra Linux и оценить эффективность реализованных механизмов мониторинга и управления.

Объект исследования – инфраструктура запуска и эксплуатации имитационных сервисов (заглушек) в среде нагрузочного тестирования.

Предмет исследования – механизмы и алгоритмы автоматизированного управления и контроля состояния имитационных сервисов в среде нагрузочного тестирования.

Научная новизна работы заключается в разработке алгоритма централизованного управления распределённой средой, поддерживающего индивидуальную настройку ограничений интенсивности запросов (Rate Limiting) для каждого имитационного сервиса в отдельности, а также в реализации агентно-независимой схемы взаимодействия с удалёнными узлами посредством протоколов SSH/SCP без установки дополнительного программного обеспечения на целевые серверы.

Практическая значимость работы состоит в создании программного инструмента, позволяющего отказаться от использования зарубежных систем управления, упростить эксплуатацию инфраструктуры имитационных сервисов и обеспечить её интеграцию со стандартными средствами мониторинга (Prometheus).

Положения, выносимые на защиту:

а) Архитектура программного комплекса, поддерживающая гибкое масштабирование системы от локального запуска на одном хосте до формирования распределённого кластера.

б) Механизм динамического проксирования запросов, реализующий прозрачную маршрутизацию входящего трафика к целевым имитационным сервисам.

в) Алгоритм контроля интенсивности запросов (Rate Limiting), обеспечивающий стабильность работы имитационной среды при пиковых нагрузках.

г) Методика формирования и экспорта метрик производительности для непрерывного мониторинга состояния инфраструктуры заглушек.

Структура работы. Выпускная квалификационная работа включает введение, три главы основного текста, заключение, список использованных источников и приложения.

Во введении обоснована актуальность исследования, поставлены цель и задачи, определены объект и предмет работы, указаны научная новизна и практическая значимость полученных результатов.

В первой главе проводится анализ предметной области, исследуются существующие аналоги и методы управления имитационными средами. На основе сравнительного анализа обосновывается выбор технологий и формируются требования к системе.

Во второй главе описывается проектирование архитектуры системы, структуры хранения данных и логики работы основных модулей. Приводятся схемы взаимодействия компонентов и алгоритмы реализации функций проксирования и управления кластером.

В третьей главе рассматриваются вопросы практической реализации, настройки и эксплуатации системы в среде ОС Astra Linux. Приводятся результаты тестирования, подтверждающие корректность работы разработанных механизмов управления и мониторинга.

В заключении сформулированы основные выводы, подтверждающие решение поставленных задач и достижение цели работы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

1.1 Исследование инфраструктуры нагрузочного тестирования и роли имитационных сервисов

В современной практике обеспечения качества сложных программных комплексов нагрузочное тестирование занимает критически важное место. Оно позволяет гарантировать стабильность и управляемость информационной системы при высоких эксплуатационных нагрузках. Одной из ключевых проблем при организации таких испытаний является необходимость взаимодействия объекта тестирования с множеством внешних систем и сервисов.

Использование реальных внешних интеграций в среде тестирования часто оказывается невозможным или нецелесообразным по ряду причин:

- ограниченность ресурсов: внешние системы могут иметь жёсткие лимиты на количество запросов или требовать значительных финансовых затрат на эксплуатацию;
- изоляция среды: для получения достоверных результатов необходимо исключить влияние задержек и сбоев сторонних систем на показатели исследуемого объекта;
- риски изменения данных: проведение тестов на реальных интеграциях может привести к нежелательному изменению или повреждению информации в продуктовых базах данных.

Для решения этих проблем применяются имитационные сервисы («заглушки») – специализированное программное обеспечение, которое воспроизводит поведение реальных компонентов, отвечая на запросы тестируемой системы заранее определённым образом. В контексте данной работы в качестве имитационных сервисов рассматриваются Java-приложения в форматах `.jar` и `.war`.

Роль имитационных сервисов в инфраструктуре нагрузочного тестирования заключается в следующем:

- обеспечение автономности стенда: заглушки позволяют полностью изолировать тестируемую систему от внешнего окружения, создавая предсказуемую среду;
- эмуляция различных сценариев: имитационные сервисы могут воспроизводить не только корректные ответы, но и ошибки (HTTP 5xx, тайм-ауты), что необходимо для проверки механизмов отказоустойчивости;
- контроль интенсивности трафика: использование механизмов ограничения запросов (Rate Limiting) позволяет моделировать поведение внешних систем при достижении ими предельных порогов производительности.

Однако при увеличении масштаба информационной системы количество необходимых заглушек возрастает, что порождает проблему управления распределённой инфраструктурой. Процессы развёртывания, контроля состояния (активна/остановлена) и маршрутизации трафика к десяткам имитационных сервисов на различных серверах в ручном режиме существенно замедляют подготовку к испытаниям.

Таким образом, для эффективного функционирования стендов нагрузочного тестирования необходима специализированная система. Она должна централизованно управлять жизненным циклом заглушек, обеспечивать их мониторинг и динамическое проксирование запросов, что особенно актуально в условиях перехода на отечественные операционные системы, такие как ОС Astra Linux.

1.2 Анализ существующих подходов к управлению сервисами-заглушками и проблема импортозамещения

Традиционным подходом к управлению имитационными сервисами (заглушками), разработанными на языке Java, является их развёртывание в специализированных контейнерах серверов приложений. В современной

практике разработки и тестирования сложился ряд стандартных решений, среди которых наиболее распространёнными являются:

- Apache Tomcat – наиболее лёгкий и популярный сервлет-контейнер, используемый для запуска веб-приложений. Он обеспечивает базовую поддержку спецификаций Java Servlet и JavaServer Pages;

- WildFly (ранее JBoss Application Server) – более мощная и полнофункциональная платформа, поддерживающая широкий спектр стандартов Java EE (Jakarta EE). Данное решение предоставляет расширенные возможности управления ресурсами и кластеризации[1];

- Eclipse GlassFish – эталонная реализация платформы Java EE, предлагающая развитые инструменты администрирования через графическую консоль и командную строку[2].

Анализ данных решений позволяет выделить общие тенденции и характеристики классических серверов приложений:

- универсальность и полнофункциональность: все перечисленные платформы спроектированы для запуска сложных корпоративных информационных систем. Однако для узкоспециализированной задачи – работы лёгковесных имитационных заглушек – их функциональность является избыточной;

- значительные накладные расходы: общим недостатком является высокое потребление оперативной памяти и ресурсов центрального процессора самой платформой. При необходимости запуска десятков или сотен заглушек на одном или нескольких хостах суммарные затраты ресурсов на поддержание среды выполнения становятся критическими;

- сложность автоматизации и оркестрации: управление большим парком разрозненных экземпляров серверов приложений требует внедрения дополнительных тяжеловесных систем автоматизации. Штатные средства управления (консоли администрирования) ориентированы на ручное конфигурирование, что затрудняет их использование в динамических сценариях нагрузочного тестирования;

— монолитная архитектура управления: классические подходы подразумевают жёсткую привязку приложения к конкретной конфигурации сервера, что снижает гибкость при необходимости быстрого изменения параметров запуска (например, JVM-аргументов) для отдельных сервисов.

На текущий момент в российской ИТ-индустрии наблюдается отчётливая тенденция к отказу от использования зарубежных проприетарных и открытых решений в пользу отечественного программного обеспечения или собственных разработок. Это обусловлено политикой импортозамещения и необходимостью обеспечения технологического суверенитета.

Большинство западных систем управления инфраструктурой и серверов приложений (таких как продукты Oracle или Red Hat) сталкиваются с проблемами поддержки, обновления и совместимости с российскими операционными системами. В частности, эксплуатация имитационных стендов в защищённой среде ОС Astra Linux требует инструментов, которые нативно поддерживают механизмы управления этой операционной системы и не зависят от зарубежных репозиторий и лицензий.

Таким образом, анализ существующих подходов показывает, что использование традиционных серверов приложений типа Tomcat или WildFly для управления распределённой сетью заглушек становится неэффективным. Современные требования диктуют необходимость перехода к более лёгким, гибким и отечественным решениям, способным обеспечить централизованное управление жизненным циклом сервисов в рамках распределённых кластеров.

1.3 Сравнительный анализ инструментальных средств разработки и обоснование технологического стека

Выбор программных средств для реализации системы управления имитационными сервисами является фундаментальным этапом проектирования. От корректности этого выбора зависят эксплуатационные характеристики бу-

дущей системы: предельная пропускная способность, латентность (время отклика), масштабируемость и устойчивость к пиковым нагрузкам.

В ходе анализа предметной области и целей исследования было определено, что разрабатываемая система должна обеспечивать бесперебойную оркестрацию интенсивного потока запросов от нагрузочных станций. Это накладывает жёсткие ограничения на архитектуру: управляющий контур не должен становиться «узким местом» (bottleneck) при маршрутизации трафика. Требование минимизации накладных расходов исключает использование технологий, склонных к блокировке вычислительных ресурсов при ожидании ответа от внешних систем.

Для обоснования выбора проводится многокритериальный анализ по трём направлениям: язык программирования, веб-фреймворк и система управления базами данных.

1.3.1 Обоснование выбора языка программирования серверной части

Для разработки backend-составляющей системы рассматривались три языка программирования, наиболее востребованных в сфере высоконагруженных систем (HighLoad) и системной автоматизации.

Java — объектно-ориентированный язык программирования, являющийся отраслевым стандартом для корпоративных систем и «родной» средой для запуска самих имитационных сервисов (.jar).

Преимущества: строгая статическая типизация, снижающая количество ошибок на этапе компиляции; развитая экосистема библиотек и инструментов профилирования; поддержка полноценной многопоточности (multithreading).

Недостатки: высокая ресурсоёмкость виртуальной машины (JVM), требующая значительного объёма оперативной памяти; длительное время «холодного старта», что критично для динамических сред тестирования; гро-

моздкий синтаксис для простых задач взаимодействия с процессами операционной системы.

Go (Golang) – компилируемый многопоточный язык программирования, разработанный Google, ориентированный на создание эффективных сетевых сервисов.

Преимущества: высочайшая производительность, сопоставимая с C++, благодаря компиляции в нативный код; эффективная модель конкурентности (goroutines), позволяющая обрабатывать тысячи соединений с минимальными накладными расходами; отсутствие внешних зависимостей (компиляция в единый бинарный файл).

Недостатки: отсутствие гибкости при работе с динамическими структурами данных (JSON произвольной структуры) из-за строгой типизации; специфическая обработка ошибок и отсутствие исключений, что увеличивает объём шаблонного кода (boilerplate); более высокий порог входа по сравнению с интерпретируемыми языками.

Python – высокоуровневый интерпретируемый язык программирования общего назначения с динамической строгой типизацией.

Преимущества: наличие мощных инструментов асинхронного программирования (библиотека `asyncio`[3]), позволяющих эффективно обрабатывать задачи, ограниченные скоростью ввода-вывода (I/O-bound), такие как сетевые запросы или ожидание ответа от базы данных; глубокая интеграция с Linux: модуль `subprocess` предоставляет удобный интерфейс для управления процессами, сигналами и потоками ввода-вывода; высокая скорость разработки благодаря лаконичному синтаксису.

Недостатки: производительность интерпретатора ниже, чем у компилируемых языков; наличие Global Interpreter Lock (GIL), ограничивающего выполнение потоков на одном ядре CPU (нивелируется использованием асинхронного подхода).

Вывод: в качестве основного языка выбран Python. Возможность использования асинхронного программирования в сочетании с нативными

средствами управления процессами Linux позволяет обеспечить необходимую производительность, сохраняя гибкость и скорость разработки.

1.3.2 Сравнительный анализ веб-фреймворков

Учитывая необходимость обеспечения высокой пропускной способности системы, выбор архитектуры веб-фреймворка является критическим. Рассматривались три кандидата: Django, Flask и FastAPI.

Django – полнофункциональный синхронный фреймворк уровня Enterprise, следующий принципу «Batteries included» (всё включено).

Преимущества: наличие встроенной панели администрирования, ORM (Object-Relational Mapping) и системы аутентификации «из коробки»; развитая экосистема плагинов и высокий уровень безопасности; строгая структура проекта, облегчающая поддержку крупных монолитных приложений.

Недостатки: монолитная архитектура и избыточный функционал создают значительные накладные расходы (overhead) на каждый запрос; синхронная модель обработки запросов плохо подходит для высоконагруженных микросервисов, так как блокирует потоки при ожидании ввода-вывода; сложность отключения неиспользуемых компонентов для облегчения системы.

Flask – лёгковесный микрофреймворк, классическое решение, основанное на спецификации WSGI (Web Server Gateway Interface).

Преимущества: минимализм и модульность: разработчик подключает только необходимые библиотеки; низкий порог входа и простота создания простых сервисов; гибкость в выборе архитектурных паттернов.

Недостатки: использование синхронной (блокирующей) модели ввода-вывода – при высокой нагрузке это приводит к исчерпанию пула потоков сервера и росту задержек; отсутствие встроенных механизмов валидации данных (требует подключения сторонних расширений); снижение производи-

тельности при большом количестве одновременных соединений (проблема C10k).

FastAPI[4] – современный асинхронный фреймворк, построенный на спецификации ASGI (Asynchronous Server Gateway Interface)[5].

Преимущества: полная поддержка асинхронности (`async/await`) и событийного цикла, что позволяет обрабатывать тысячи запросов в секунду на одном процессе; высокая производительность благодаря использованию Starlette и uvloop, сопоставимая с Go и Node.js; встроенная быстрая валидация данных и сериализация на базе Pydantic[6].

Недостатки: относительно молодая экосистема по сравнению с Django и Flask; требует понимания принципов асинхронного программирования.

Вывод: выбран фреймворк FastAPI [4], [7]. Это единственное решение в экосистеме Python, способное обеспечить стабильную обработку интенсивного трафика с минимальными задержками благодаря асинхронной неблокирующей архитектуре.

1.3.3 Обоснование выбора системы управления данными (СУБД)

Для обеспечения высокой отзывчивости системы подсистема хранения данных должна обладать минимальным временем отклика на чтение конфигураций и запись статусов. Сравнивались конкретные распространённые решения.

PostgreSQL – объектно-реляционная СУБД с открытым исходным кодом.

Преимущества: гарантия целостности данных (ACID) и надёжность хранения; мощный язык запросов SQL и поддержка сложных выборок (JOIN); широкие возможности расширяемости (типы данных, индексы).

Недостатки: накладные расходы на дисковые операции ввода-вывода и журналирование (WAL) замедляют отклик; избыточность функционала для

хранения простых пар «ключ-значение» (конфигурации заглушек); потребность в строгом проектировании схемы данных (Schema migrations).

MySQL – реляционная СУБД, популярная база данных, часто используемая в веб-приложениях.

Преимущества: высокая скорость операций чтения в простых сценариях; широкая распространённость и простота администрирования; поддержка различных движков хранения (Storage Engines).

Недостатки: как и любая реляционная СУБД, зависит от скорости дисковой подсистемы; блокировки таблиц или строк могут снижать конкурентность записи при высокой нагрузке; жёсткость схемы данных.

MongoDB – документоориентированная NoSQL СУБД, система, хранящая данные в формате BSON (бинарный JSON).

Преимущества: гибкая схема данных (Schemaless), идеально подходящая для хранения JSON-конфигураций; горизонтальная масштабируемость (шардинг) из коробки.

Недостатки: потребляет значительно больше памяти и дискового пространства по сравнению с аналогами; в стандартной конфигурации уступает In-Memory решениям по скорости отклика; слабые гарантии транзакционности в старых версиях.

Redis – In-Memory хранилище структур данных[8], высокопроизводительная СУБД, работающая полностью в оперативной памяти.

Преимущества: экстремально низкая латентность (время доступа менее 1 мс) благодаря отсутствию обращений к диску при чтении; простая модель данных «Ключ-Значение», полностью соответствующая задаче хранения состояний процессов; нативная поддержка механизма Time-To-Live (TTL) для временных данных.

Недостатки: зависимость объёма хранимых данных от объёма оперативной памяти; риск потери данных при аварийном отключении питания (решается настройкой персистентности).

Вывод: выбрана СУБД Redis[8]. Использование In-Memory архитектуры обеспечивает максимальное быстроедействие. Проблема сохранности данных решается включением режима AOF (Append Only File)[9], который асинхронно сохраняет операции на диск, гарантируя восстановление конфигураций после перезагрузки без ущерба для производительности.

1.3.4 Итоговый выбор средств разработки

На основании проведённого анализа и выявленных требований к быстроедействию и надёжности утверждён следующий стек технологий для реализации выпускной квалификационной работы:

- язык программирования: Python (с использованием `asyncio`);
- веб-фреймворк: FastAPI (сервер приложений Uvicorn);
- СУБД: Redis (режим персистентности AOF);
- целевая платформа: ОС Astra Linux Special Edition[10].

Данный выбор обеспечивает оптимальный баланс между производительностью, скоростью разработки и надёжностью функционирования системы в условиях высокой нагрузки.

1.4 Обзор аналогичных решений и обоснование разработки собственного программного комплекса

В рамках анализа предметной области необходимо рассмотреть существующие на рынке инструменты, применяемые для создания, эксплуатации и управления имитационными сервисами (заглушками). Поскольку разрабатываемая система предназначена для управления инфраструктурой заглушек в процессе нагрузочного тестирования, к аналогам следует отнести программные продукты, обеспечивающие виртуализацию сервисов (Service Virtualization), мокирование (Mocking), а также серверы приложений, традиционно используемые для хостинга Java-артефактов.

Сравнительный анализ проводится по критериям, критически важным для обеспечения стабильности нагрузочного стенда и автоматизации процессов тестирования:

- архитектура и потребление ресурсов: способность работать в условиях ограниченных аппаратных мощностей при запуске десятков экземпляров заглушек;

- возможности управления процессами (Orchestration): наличие инструментов для удалённого запуска, остановки и обновления версий приложений-заглушек;

- функциональность управления трафиком: наличие встроенных механизмов ограничения нагрузки (Rate Limiting) и эмуляции сетевых задержек;

- совместимость и импортозамещение: возможность автономной работы в среде ОС Astra Linux без зависимости от зарубежных облачных сервисов и проприетарных лицензий.

1.4.1 Анализ решения WireMock

WireMock – один из наиболее распространённых инструментов с открытым исходным кодом для виртуализации HTTP-сервисов. Продукт позволяет создавать заглушки, настраивать правила сопоставления запросов (request matching) и формировать динамические ответы.

Согласно официальной документации [11], WireMock поддерживает гибкую настройку ответов через JSON API, имитацию задержек (network latency) и ошибок (fault injection). Он может работать в двух режимах: как библиотека, встраиваемая в код автотестов, или как отдельный процесс (Standalone).

Недостатки в контексте поставленной задачи:

- ресурсоёмкость: WireMock разработан на Java и выполняется в виртуальной машине JVM. Запуск отдельного экземпляра WireMock для каждого имитируемого микросервиса создаёт значительные накладные расходы на

оперативную память (RAM) и процессорное время (CPU). В условиях нагрузочного тестирования, когда требуется поднять 50–100 заглушек, суммарные затраты ресурсов на поддержание работы самих заглушек становятся неприемлемыми;

- отсутствие встроенной оркестрации: в бесплатной версии (Open Source) WireMock отсутствует централизованная панель управления распределённым кластером. Инструмент фокусируется на логике ответа, а не на эксплуатации сервера;

- ограничения Rate Limiting: базовый функционал позволяет имитировать задержки, однако полноценный алгоритм ограничения количества запросов в секунду (RPS) с отбрасыванием лишнего трафика требует написания кастомных расширений (extensions) на Java [11].

1.4.2 Анализ решения MockServer

MockServer – популярное решение для мокирования систем, взаимодействующих по протоколам HTTP или HTTPS. Продукт часто используется в контейнеризированных средах (Docker, Kubernetes) для интеграционного тестирования.

MockServer предоставляет мощный API для создания «ожиданий» (Expectations) и верификации запросов. Согласно документации [12], он поддерживает проксирование трафика (Port Forwarding), запись проходящих запросов для последующего анализа и генерацию ответов на основе шаблонов (Mustache, Velocity).

Недостатки в контексте поставленной задачи:

- монолитная архитектура управления: MockServer хранит состояния «ожиданий» в памяти конкретного экземпляра приложения. Для создания распределённой системы требуется сложная настройка кластеризации, что утяжеляет инфраструктуру тестового стенда;

— сложность динамического управления: MockServer ориентирован на программное управление через API во время прогона тестов. Использование его как постоянно действующего инфраструктурного компонента требует разработки дополнительной обвязки для мониторинга его состояния и автоматического перезапуска (Self-healing) [12];

— зависимость от Java: аналогично WireMock, продукт написан на Java, что влечёт проблемы с «холодным стартом» и высоким потреблением памяти (Heap Size), снижающим плотность размещения заглушек на сервере.

1.4.3 Анализ решения Mountebank

Mountebank – инструмент с открытым исходным кодом, разработанный на платформе Node.js. Его ключевой особенностью является мультипротокольность: помимо HTTP/HTTPS, он поддерживает TCP, SMTP и другие протоколы через систему плагинов.

Mountebank использует концепцию «самозванцев» (imposters) – лёгких сервисов, слушающих определённые порты. Документация [13] указывает на возможность использования JavaScript-инъекций для реализации сложной логики обработки ответов.

Недостатки в контексте поставленной задачи:

— проблемы безопасности: возможность исполнения произвольного JS-кода (injection) в теле запроса создаёт уязвимости в контуре нагрузочного тестирования;

— отсутствие управления артефактами: Mountebank позволяет создать логику ответа (скрипт), но не предназначен для запуска и управления бинарными файлами (заглушками в виде .jar), предоставленными разработчиками;

— производительность в однопоточном режиме: в стандартной конфигурации Mountebank работает в одном потоке (Event Loop). При высокой нагрузке (тысячи RPS), характерной для нагрузочного тестирования, сложные

вычисления внутри одной заглушки могут заблокировать обработку запросов других заглушек, работающих в том же экземпляре Mountebank [13].

1.4.4 Анализ решения Apache Tomcat

Apache Tomcat – свободно распространяемый контейнер сервлетов, реализующий спецификации Jakarta Servlet и Jakarta Pages. Это классическое решение для хостинга Java-приложений, широко применяемое в промышленной эксплуатации.

Согласно официальной документации [14], Tomcat предоставляет развитые средства управления жизненным циклом веб-приложений. Компонент Manager App позволяет развёртывать (deploy), останавливать и перезапускать приложения без остановки сервера. Поддерживается кластеризация для репликации сессий [15]. Для защиты от перегрузок предусмотрен фильтр RateLimitFilter, позволяющий ограничивать количество запросов с одного IP-адреса [16].

Недостатки в контексте поставленной задачи:

- избыточность архитектуры (Overhead): Tomcat является полноценным сервлет-контейнером. Использование его для запуска примитивных заглушек нерационально с точки зрения потребления ресурсов. Запуск отдельного экземпляра Tomcat для каждой заглушки потребляет сотни мегабайт оперативной памяти только на нужды самого сервера, что критично при дефиците ресурсов;

- требования к формату артефактов: Tomcat работает с WAR-архивами. Однако современные микросервисы (и заглушки для них) чаще всего поставляются в виде Fat JAR (со встроенным сервером, например, Netty или Undertow). Для запуска таких заглушек в Tomcat требуется их пересборка и модификация кода, что усложняет процесс;

- статичность конфигурации ограничений: механизмы Rate Limiting в Tomcat конфигурируются декларативно через XML-файлы при старте. В за-

дачах нагрузочного тестирования требуется динамическое изменение параметров дросселирования трафика «на лету» (runtime) без перезагрузки сервера, что в Tomcat не реализовано «из коробки».

1.4.5 Сравнительная характеристика

Для наглядного сравнения рассмотренных решений с проектируемой системой составлена таблица 1.

Таблица 1 – Сравнительный анализ средств управления имитационными сервисами

Критерий	WireMock	Mock-Server	Mountebank	Apache Tomcat	Разраб. система
Управление JAR/WAR без пересборки	Нет	Нет	Нет	Частично	Да
Централизованная оркестрация	Нет	Нет	Нет	Нет	Да
Динамический Rate Limiting	Нет	Нет	Нет	Нет	Да
Потребление ресурсов	Высокое (JVM)	Высокое (JVM)	Среднее	Высокое (JVM)	Низкое
Совместимость с Astra Linux	Частично	Частично	Да	Частично	Да

1.4.6 Обоснование разработки собственного программного комплекса

Проведённый анализ показывает, что существующие на рынке решения делятся на две категории:

— инструменты логической виртуализации (WireMock, MockServer, Mountebank) – отлично справляются с задачей «что ответить на запрос»,

но не решают задачу управления инфраструктурой (запуск, перезапуск, обновление версий приложений);

— серверы приложений (Apache Tomcat) – умеют управлять жизненным циклом, но слишком тяжеловесны, требуют специфического формата упаковки (WAR) и сложны в динамической настройке сетевых параметров.

Ни одно из рассмотренных решений не предоставляет функционала «лёгковесного агента» для управления сторонними Java-процессами «как есть» (без перепакетки).

Разработка собственной автоматизированной информационной системы обоснована следующими факторами.

Специфика задачи (Lifecycle Management): разрабатываемая система решает задачу оркестрации процессов операционной системы. Она позволяет загружать бинарный файл заглушки (JAR), полученный от разработчиков, и управлять его исполнением на удалённом сервере. Это исключает трудозатраты на пересборку заглушек под WireMock или Tomcat.

Эффективность использования ресурсов: перенос логики управления на Python и использование асинхронного фреймворка FastAPI позволяет минимизировать накладные расходы самого управляющего контура. Система работает поверх заглушек, не внедряясь в их память, в отличие от тяжёлых Java-контейнеров.

Централизация и удобство эксплуатации: реализация архитектуры «Master-Agent» объединяет разрозненные серверы в единый кластер. Единая панель управления (Dashboard) позволяет инженеру тестирования массово обновлять версии заглушек и менять настройки лимитов (RPS) в реальном времени, что невозможно при использовании Standalone-решений.

Технологическая независимость и импортозамещение: система разрабатывается на базе открытых технологий, адаптирована для работы в ОС Astra Linux и не зависит от внешних репозиторий (Maven Central, Docker Hub) или лицензионных ключей, что гарантирует стабильность работы в закрытых корпоративных контурах РФ.

1.4.7 Вывод по подразделу

Разработка собственного программного комплекса является единственным целесообразным решением, закрывающим пробел между инструментами логического мокирования и средствами управления инфраструктурой, обеспечивая высокую производительность и гибкость конфигурации тестовых сред.

1.5 Формирование функциональных и технических требований к системе

На основании проведённого анализа предметной области и специфики задач нагрузочного тестирования сформированы требования к разрабатываемому программному комплексу. Система классифицируется как инструмент автоматизации инфраструктуры нагрузочного тестирования. Архитектура решения должна быть гибкой, допускающей работу как в распределённом контуре, так и в рамках одного физического или виртуального сервера.

1.5.1 Требования к режимам функционирования

Система должна поддерживать два сценария развёртывания без изменения кодовой базы:

- распределённый режим (Cluster Mode): классическая клиент-серверная архитектура, где управляющий узел (Master) администрирует множество удалённых узлов (Agents), на которых выполняются заглушки. Взаимодействие осуществляется по сети;

- автономный режим (Standalone Mode): консолидированный запуск всех компонентов системы на одном сервере. В этом режиме управляющий модуль и модуль исполнения заглушек функционируют в пределах одной операционной системы, обеспечивая изоляцию контура тестирования без необходимости сетевого взаимодействия между узлами инфраструктуры.

1.5.2 Функциональные требования

Функциональные требования определяют перечень задач, которые система должна выполнять для обеспечения процессов эмуляции сервисов.

1.5.2.1 Требования к подсистеме управления жизненным циклом (Runtime)

- поддержка форматов артефактов: система должна обеспечивать запуск Java-приложений, поставляемых в форматах исполняемых файлов `.jar` и веб-архивов `.war`;
- управление процессами: обеспечение запуска, корректной остановки и принудительного завершения процессов заглушек;
- конфигурация запуска: возможность задания аргументов виртуальной машины (JVM Options), включая параметры памяти (`-Xms`, `-Xmx`) и системные свойства, через графический интерфейс управляющего узла (Master) перед стартом заглушки;
- мониторинг: отслеживание статуса процесса (PID) и доступности его сетевого порта;
- работа с логами: сбор стандартных потоков вывода (`stdout`, `stderr`) запущенных процессов и их трансляция в интерфейс системы для отладки.

1.5.2.2 Требования к подсистеме проксирования запросов (Proxy Engine)

Система должна работать как прозрачный прокси-сервер (Transparent Proxy) на прикладном уровне, реализуя следующий алгоритм:

- а) приём входящего HTTP-запроса от внешней системы;
- б) извлечение метаданных: HTTP-метод, заголовки, тело запроса и целевой путь (URI);

в) выполнение нового запроса к целевой заглушке (на её локальный порт) с использованием извлечённого метода, заголовков и тела;

г) получение ответа от заглушки;

д) возврат полученного ответа инициатору запроса в неизменном виде.

Требования к управлению нагрузкой (Rate Limiting):

— наличие механизма ограничения количества запросов в секунду (RPS) для конкретной заглушки;

— активация лимита производится через вызов конфигурационного энд-поинта системы (API). Динамическое изменение алгоритма ограничения в процессе теста не требуется;

— при превышении лимита система должна возвращать HTTP-код 429 (Too Many Requests), не передавая запрос заглушке.

1.5.2.3 Требования к управляющему модулю (Master/Dashboard)

— управление артефактами: загрузка бинарных файлов (.jar, .war) в хранилище системы через веб-интерфейс;

— реестр сервисов: отображение списка всех загруженных заглушек, их текущего статуса («Запущен», «Остановлен», «Ошибка») и адресов;

— интерфейс конфигурации: предоставление полей ввода для настройки параметров запуска (JVM args) для каждого сервиса.

1.5.3 Нефункциональные (технические) требования

1.5.3.1 Требования к производительности

— вносимая задержка (Overhead): накладные расходы на проксирование (время обработки запроса внутри системы без учёта времени работы самой заглушки) не должны превышать 20 мс;

— пропускная способность: система должна обеспечивать высокую производительность проксирующего слоя. Снижение максимальной пропускной

способности (RPS) заглушки при работе через систему не должно превышать 10% относительно прямого обращения к заглушке.

1.5.3.2 Требования к надёжности

— сохранение конфигурации и восстановление работоспособности: при перезапуске системы (или сервера) должны сохраняться настройки запуска заглушек и ссылки на загруженные артефакты (Persistence); для заглушек, находившихся в активном состоянии до перезапуска, система должна предпринять автоматическую попытку повторного запуска процесса;

— изоляция сбоев: ошибка в одной заглушке не должна влиять на работоспособность остальных запущенных сервисов и самой управляющей системы.

1.5.4 Требования к программно-аппаратному обеспечению

1.5.4.1 Системные требования

— операционная система: серверная часть должна функционировать под управлением ОС Astra Linux Special Edition (версия 1.7 и выше) или аналогичных Linux-дистрибутивов;

— среда исполнения Java: поддержка OpenJDK 17[17] для запуска пользовательских артефактов.

1.5.4.2 Стек разработки

— язык программирования: Python (версия 3.10+) с использованием асинхронного подхода;

— веб-фреймворк: FastAPI (для реализации высокопроизводительного ввода-вывода);

— СУБД: Redis (для хранения конфигураций и реализации счётчиков Rate Limiter).

1.5.5 Требования к информационной безопасности

— контроль доступа: доступ к административному интерфейсу и API управления должен быть ограничен;

— безопасность данных: загружаемые исполняемые файлы должны храниться в изолированных директориях с правами доступа, исключающими их несанкционированную модификацию третьими лицами.

1.6 Выводы по первой главе

В первой главе выпускной квалификационной работы был проведён комплексный анализ предметной области и обоснована необходимость создания специализированного инструментария для автоматизации нагрузочного тестирования.

В ходе исследования выявлено, что использование традиционных серверов приложений и существующих аналогов (WireMock, MockServer, Mountebank) для управления распределённой сетью заглушек сопряжено с избыточным потреблением ресурсов и сложностями в эксплуатации, а также не удовлетворяет требованиям по импортозамещению.

На основании сравнительного анализа обоснован выбор технологического стека: язык программирования Python с использованием асинхронного подхода, веб-фреймворк FastAPI для обеспечения высокой пропускной способности и In-Memory СУБД Redis для хранения состояний с минимальной задержкой.

Определена монолитная архитектура будущего решения с управлением удалёнными хостами через SSH/SCP и сформированы функциональные и технические требования к системе, включая поддержку форматов `.jar` и `.war`, механизмы ограничения нагрузки (Rate Limiting) и совместимость с

ОС Astra Linux. Полученные результаты служат базисом для проектирования и технической реализации системы во второй главе.

2 ПРОЕКТИРОВАНИЕ И ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ УПРАВЛЕНИЯ

2.1 Разработка архитектуры распределённого программного комплекса

2.1.1 Выбор архитектурного стиля

Проектирование архитектуры является фундаментальным этапом разработки программного комплекса: именно на данной стадии закладываются структурные свойства системы, определяющие её масштабируемость, надёжность, сопровождаемость и производительность в долгосрочной перспективе. В рамках настоящего раздела проводится анализ применимых архитектурных подходов, обосновывается выбор итогового архитектурного решения и формализуется структура разработанного программного комплекса.

В современной инженерии программного обеспечения наибольшее распространение получили два конкурирующих архитектурных стиля: монолитная и микросервисная архитектуры. Для обоснованного выбора необходимо рассмотреть их характеристики в контексте требований, сформулированных в первой главе настоящей работы.

Микросервисная архитектура предполагает декомпозицию системы на множество независимо разворачиваемых сервисов, каждый из которых реализует конкретную бизнес-функцию и взаимодействует с остальными посредством сетевых протоколов – как правило, HTTP REST или gRPC. Данный подход демонстрирует преимущества в условиях работы крупных распределённых команд разработчиков, при необходимости независимого масштабирования отдельных компонентов и при высокой частоте выпуска обновлений изолированных частей системы. К характерным достоинствам микросервисной архитектуры относятся: независимость технологического стека отдельных сервисов, устойчивость к отказам на уровне изолированных процессов,

а также возможность горизонтального масштабирования конкретных узлов без перезапуска всей системы.

Вместе с тем микросервисный подход сопряжён с существенными эксплуатационными издержками, принципиально важными в контексте настоящего проекта. Во-первых, каждый сервис требует собственной инфраструктуры развёртывания: отдельный сетевой порт, конфигурационный файл, процесс инициализации и мониторинга. Во-вторых, взаимодействие сервисов по сети порождает дополнительные задержки и накладные расходы на сериализацию и десериализацию данных при каждом межсервисном вызове. В-третьих, полноценная реализация микросервисной архитектуры требует развёртывания дополнительных инфраструктурных компонентов: системы оркестрации контейнеров, сервисного реестра и системы трассировки запросов. Перечисленные компоненты существенно увеличивают сложность системы и создают зависимости от зарубежного программного обеспечения, что противоречит требованию технологической независимости, сформулированному в первой главе.

Монолитная архитектура объединяет все компоненты системы в рамках единого процесса операционной системы, разделяя общее адресное пространство памяти, единую точку входа и общую конфигурацию развёртывания. Несмотря на устойчивое представление о монолите как об устаревшем архитектурном решении, данный подход демонстрирует высокую эффективность в условиях, характерных для разрабатываемого комплекса: команда разработки состоит из одного специалиста, функциональные требования чётко определены и стабильны, а потребности в раздельном масштабировании отдельных внутренних компонентов отсутствуют.

Ключевым аргументом в пользу монолитного подхода является специфика решаемой задачи: разрабатываемая система является «тонким» координирующим слоем над управляемыми имитационными сервисами. Её основная функция – оркестрация процессов операционной системы на целевых хостах, маршрутизация входящего трафика и хранение конфигурационных дан-

ных. Введение внутрисистемных сетевых вызовов в рамках гипотетической микросервисной декомпозиции создало бы накладные расходы, несоизмеримые выигрышу от разделения компонентов.

Распределённость системы, заявленная в названии раздела, реализуется не на уровне внутреннего устройства приложения, а на уровне топологии развёртывания: единый управляющий процесс координирует целевые хосты посредством защищённых сетевых протоколов. Таким образом, распределённость является внешней характеристикой инфраструктуры, а не принципом внутренней декомпозиции.

На основании проведённого анализа для реализации программного комплекса выбрана монолитная архитектура с явно выраженной внутренней модульной структурой. Данное решение обеспечивает минимальные накладные расходы на взаимодействие компонентов, простоту развёртывания и эксплуатации в закрытом корпоративном контуре, технологическую независимость от внешних оркестраторов, а также высокую сопровождаемость и читаемость единой кодовой базы.

В таблице 2 приведён сравнительный анализ двух рассмотренных архитектурных подходов по критериям, наиболее значимым для данного проекта.

Таблица 2 – Сравнение архитектурных подходов

Критерий	Монолитная архитектура	Микросервисная архитектура
Накладные расходы	Минимальные	Высокие
Сложность развёртывания	Низкая	Высокая
Зависимость от внешних систем	Отсутствует	Требуется оркестратор
Сопровождаемость	Высокая	Низкая
Масштабирование компонентов	Не поддерживается	Поддерживается

2.1.2 Топология развёртывания и гибкость инфраструктуры

Одним из ключевых проектных решений, определивших структуру системы, является принцип гибкого размещения имитационных сервисов. В отличие от систем, жёстко делящихся на «локальный» и «кластерный» режимы, разработанный программный комплекс предоставляет оператору возможность назначать произвольный целевой хост для каждой заглушки независимо.

Распределённость системы реализуется посредством реестра хостов: каждый зарегистрированный хост описывается своим сетевым адресом, учётной записью для подключения и рабочей директорией. При развёртывании конкретной заглушки оператор выбирает один из хостов реестра в качестве целевого. Это позволяет в рамках единого экземпляра управляющего приложения одновременно:

- размещать часть заглушек непосредственно на сервере, где работает управляющее приложение (адрес `127.0.0.1` или `hostname` сервера), – что соответствует традиционному «локальному» сценарию;
- размещать другие заглушки на удалённых серверах через SSH/SCP – что соответствует «кластерному» сценарию;
- совмещать оба варианта в рамках одной инфраструктуры без каких-либо изменений в конфигурации приложения.

Таким образом, граница между «локальным» и «распределённым» режимами определяется не конфигурацией запуска приложения, а составом реестра хостов и выбором оператора при регистрации каждой заглушки. Такой подход значительно увеличивает гибкость эксплуатации системы и позволяет плавно наращивать инфраструктуру без вмешательства в работу уже запущенных сервисов.

С точки зрения внутренней архитектуры это означает, что модуль управления жизненным циклом всегда работает по единому алгоритму, независимо от того, является ли целевой хост локальным или удалённым. При

работе с `localhost` SSH-соединение устанавливается к локальному SSH-серверу по стандартному порту 22, что унифицирует код и устраняет необходимость в двух ветках логики для локального и удалённого сценариев.

2.1.3 Контекстная диаграмма системы

Для формализованного описания архитектуры программного комплекса применяется нотация C4 Model[18], предусматривающая четыре уровня детализации: контекстный (Level 1), контейнерный (Level 2), компонентный (Level 3) и кодовый (Level 4). Нотация обеспечивает однозначность интерпретации архитектурных решений и соответствует требованиям методических указаний по применению стандартизованных схем для представления структуры системы.

Контекстная диаграмма (рисунок 1) отображает систему как единый «чёрный ящик» в окружении внешних акторов и смежных систем. На данном уровне абстракции детали внутреннего устройства системы не раскрываются; основное внимание уделяется определению границ системы и характеру её взаимодействия с внешним окружением.

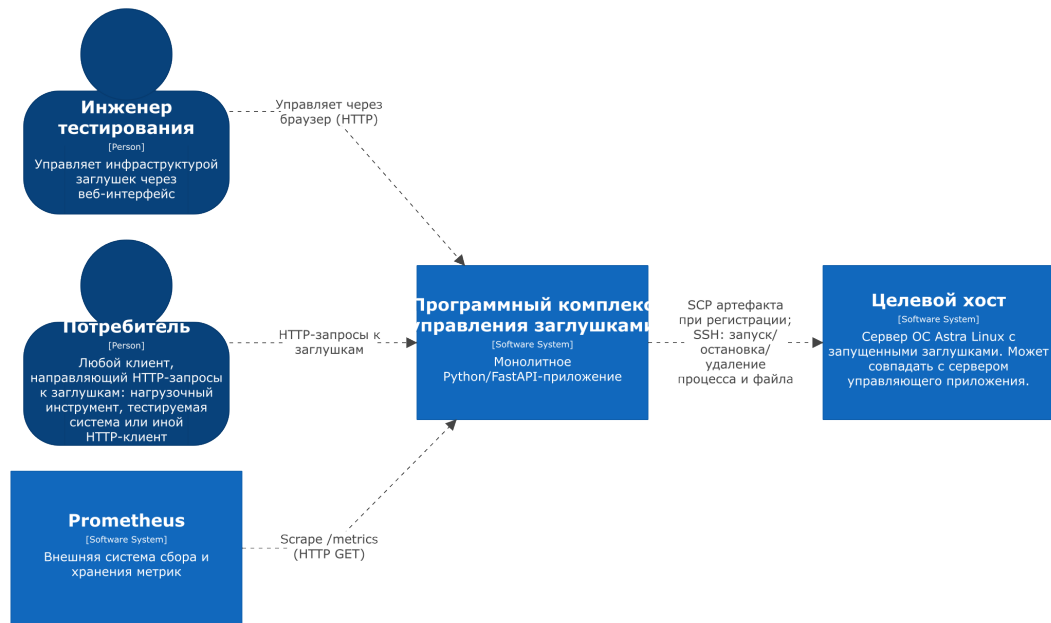


Рисунок 1 – Контекстная диаграмма системы (C4 Level 1)

Внешними акторами системы являются:

- инженер тестирования – специалист, осуществляющий управление инфраструктурой имитационных сервисов через веб-интерфейс системы; выполняет операции регистрации артефактов, запуска и остановки заглушек, настройки параметров и просмотра метрик;
- потребитель – любой HTTP-клиент, направляющий запросы к имитационным сервисам через прокси-движок системы: это может быть инструмент нагрузочного тестирования (Apache JMeter, Gatling и др.), тестируемая система (System Under Test) или иной HTTP-клиент;
- Prometheus – внешняя система сбора и хранения метрик, периодически опрашивающая специализированный эндпоинт системы для получения телеметрических данных;
- целевой хост – сервер под управлением ОС Astra Linux[10], на котором непосредственно выполняются Java-процессы имитационных сервисов[17]; может совпадать с сервером управляющего приложения.

2.1.4 Контейнерная диаграмма

Контейнерная диаграмма (рисунок 2) раскрывает внутреннее устройство программного комплекса на уровне самостоятельно исполняемых программных единиц – контейнеров. В терминологии C4 Model «контейнер» обозначает логически обособленную программную единицу, а не технологию Docker-контейнеризации. Для каждого контейнера указывается его технологический стек и зона ответственности.

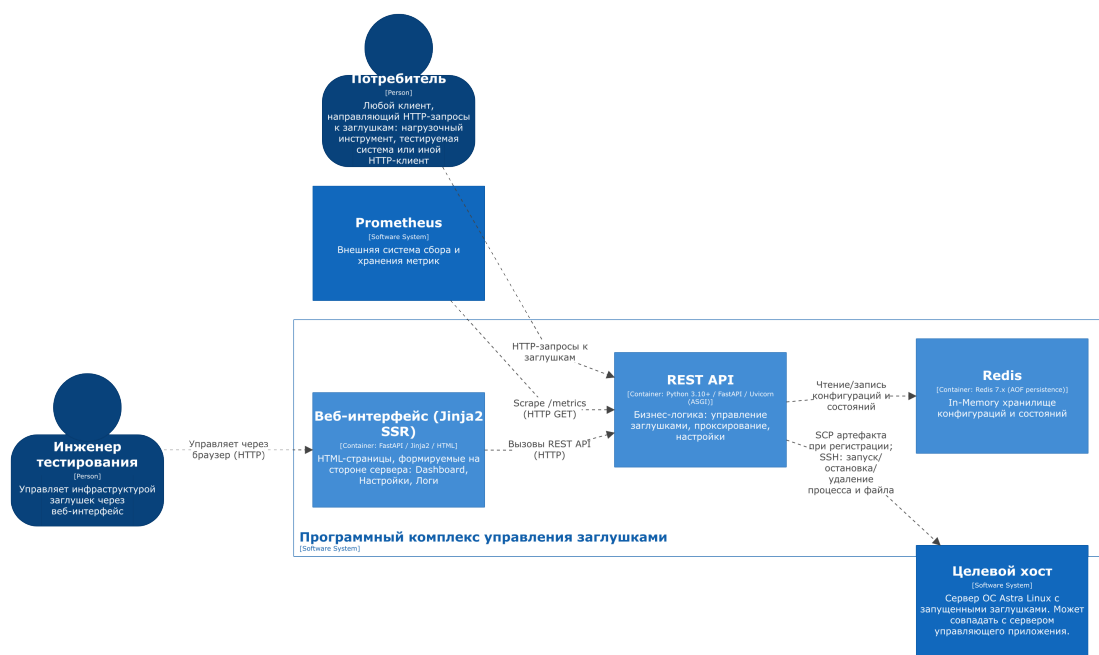


Рисунок 2 – Контейнерная диаграмма системы (C4 Level 2)

В составе программного комплекса выделяются следующие контейнеры.

Веб-интерфейс (Jinja2 SSR[19]). Реализует пользовательский интерфейс на основе технологии Server-Side Rendering с использованием шаблонизатора Jinja2, встроенного в экосистему FastAPI. HTML-страницы формируются на стороне сервера и передаются браузеру в готовом виде. Данный подход минимизирует объём JavaScript-кода на стороне клиента и обеспечивает полную функциональность интерфейса без зависимости от внешних CDN. Веб-интерфейс включает: главную панель (Dashboard) со списком загрузок,

страницу управления хостами и настроек системы, страницу просмотра журналов процессов.

REST API (FastAPI). Центральный контейнер системы, реализующий всю бизнес-логику. Запускается под управлением ASGI-сервера Uvicorn. REST API предоставляет конечные точки для взаимодействия с веб-интерфейсом, а также программный интерфейс для возможной интеграции со сторонними системами автоматизации. Внутренняя структура контейнера детализируется на компонентном уровне в разделе 2.1.5.

Redis. In-Memory хранилище структур данных, выполняющее функции постоянного хранилища конфигурационных данных системы и оперативно-го хранилища счётчиков алгоритма ограничения запросов. Redis функционирует в режиме персистентности AOF (Append Only File), что обеспечивает сохранность данных при аварийном перезапуске управляющей системы. Детальное описание структур данных Redis приводится в разделе 2.2.

Система намеренно не содержит выделенного контейнера хранилища артефактов. Архитектурное решение «1 заглушка – 1 хост» исключает необходимость повторного развёртывания одного артефакта на несколько хостов, а значит, постоянное хранение загруженного файла на сервере управляющего приложения избыточно. При регистрации заглушки её бинарный файл принимается из браузера во временный каталог, немедленно копируется на целевой хост посредством SCP и удаляется с сервера управляющего приложения. Единственной постоянной копией артефакта остаётся файл на целевом хосте в рабочей директории, указанной в реестре хостов.

2.1.5 Компонентная диаграмма REST API

Компонентная диаграмма (рисунок 3) детализирует внутреннее устройство контейнера REST API, раскрывая составляющие его программные модули – компоненты.

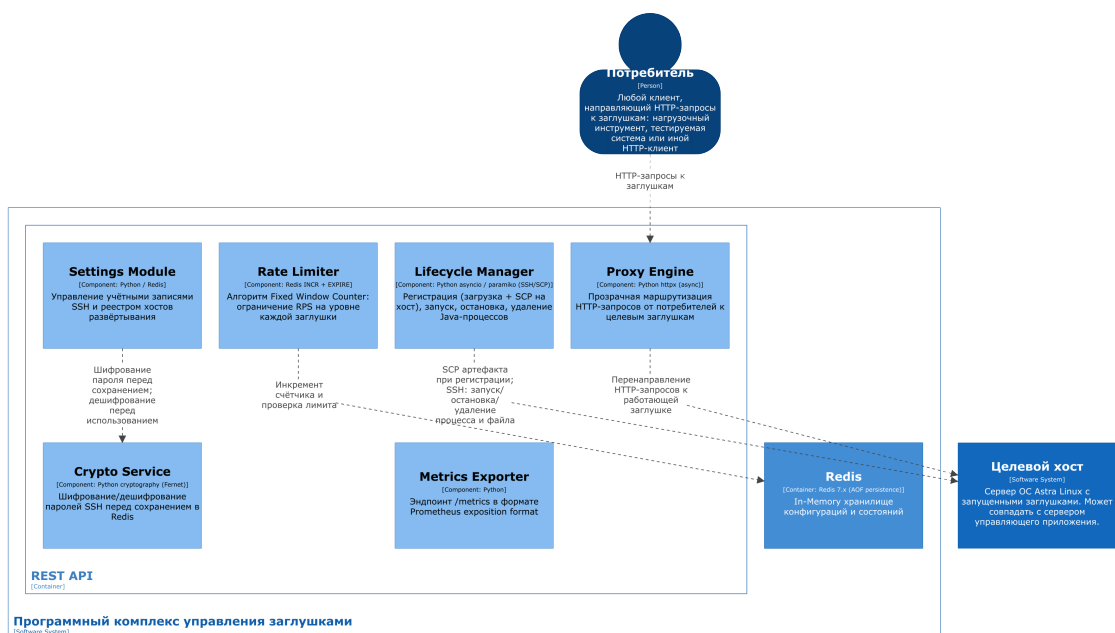


Рисунок 3 – Компонентная диаграмма REST API (C4 Level 3)

Lifecycle Manager. Реализует операции управления жизненным циклом имитационных сервисов. При регистрации новой заголовки модуль принимает загруженный бинарный файл во временный каталог на сервере управляющего приложения, копирует его на целевой хост посредством SCP в рабочую директорию, указанную в записи хоста, и немедленно удаляет временный файл. Путь к файлу на целевом хосте не сохраняется в Redis – он вычисляется при необходимости как конкатенация поля `working_dir` из записи `hosts:{hostname}` и имени файла заголовки. При запуске модуль устанавливает SSH-соединение с целевым хостом[20], определяет свободный порт в заданном диапазоне и инициирует Java-процесс с указанными JVM-аргументами в фоновом режиме (`nohup`). При остановке выполняется корректное завершение процесса через SSH. При удалении заголовки производятся: остановка процесса (если запущен), удаление бинарного файла с целевого хоста командой `SSH rm` и удаление записи из Redis.

Proxy Engine. Реализует функциональность прозрачного обратного прокси-сервера[21] на прикладном уровне. При поступлении HTTP-запроса движок извлекает адрес и порт целевой заголовки из Redis, формирует но-

вый HTTP-запрос с сохранением метода, заголовков и тела оригинального запроса, а затем возвращает ответ заглушки инициатору в неизменном виде. Для асинхронного выполнения исходящих запросов используется библиотека `httpx`[21].

Rate Limiter. Реализует алгоритм «Счётчик с фиксированным временным окном» (Fixed Window Counter). Компонент функционирует на уровне каждой заглушки независимо и управляется флагом `rate_limit_enabled`, переключаемым оператором через Dashboard без остановки заглушки. Детальное описание алгоритма приводится в разделе 2.4.

Settings Module. Обеспечивает управление конфигурационными параметрами системы – учётными записями ОС Linux и реестром хостов развёртывания. Подробное описание приведено в разделе 2.1.6.

Crypto Service. Выполняет шифрование паролей SSH перед записью в Redis и дешифрование перед использованием для установки соединения. Применяется алгоритм AES-128 (Fernet [22]). Ключ шифрования хранится на файловой системе сервера в файле с правами 600.

Metrics Exporter. Реализует эндпоинт `/metrics` в формате Prometheus exposition format[23], [24]. Агрегирует данные о RPS, срабатываниях Rate Limiter, статусах процессов и доступности хостов. Детальное описание приведено в разделе 2.6.

2.1.6 Архитектура модуля настроек системы

Модуль настроек (Settings Module) предоставляет администратору системы веб-интерфейс для конфигурирования двух категорий параметров: учётных записей операционной системы Linux, применяемых для SSH/SCP-операций, и реестра хостов, доступных для развёртывания имитационных сервисов.

Управление учётными записями ОС Linux. Данная категория позволяет зарегистрировать именованные учётные записи пользователей операци-

онной системы Linux. Каждая учётная запись используется для аутентификации при установке SSH-соединения с целевым хостом и включает имя пользователя (login) и пароль. Применение отдельных учётных записей с ограниченными правами доступа обусловлено требованиями информационной безопасности: рабочая директория заглушек должна быть доступна только указанному пользователю, что минимизирует риски при нештатном поведении Java-процессов.

Механизм защищённого хранения паролей. Поскольку пароли учётных записей SSH хранятся в Redis, необходимо обеспечить их конфиденциальность в случае несанкционированного доступа к базе данных. Для этого применяется симметричное шифрование на основе алгоритма AES-128 в режиме CBC, реализованного в библиотеке `cryptography` пакетом `Fernet`. Схема работы следующая:

- при первом запуске приложения генерируется или загружается из защищённого конфигурационного файла ключ шифрования (Fernet key), который хранится вне Redis – в файле с правами доступа 600, доступном только пользователю, от имени которого запущено приложение;

- перед сохранением пароля в Redis компонент `Crypto Service` шифрует его с использованием Fernet-ключа[22] и сохраняет в виде строки Base64-кодированного зашифрованного блока;

- перед использованием пароля для установки SSH-соединения `Crypto Service` выполняет дешифрование, получая исходный пароль в открытом виде только в оперативной памяти процесса на время установки соединения.

Такой подход гарантирует, что даже при получении злоумышленником дампа Redis пароли останутся недоступными без Fernet-ключа, хранящегося на файловой системе сервера.

Реестр хостов развёртывания. Данная категория настроек представляет собой список серверов, доступных для развёртывания имитационных сервисов. Каждая запись реестра описывает конкретный целевой хост. В таб-

лице 3 приведено описание параметров модуля настроек с указанием типов данных и ограничений.

Таблица 3 – Параметры модуля настроек системы

Параметр	Тип	Описание
hostname	String	IP-адрес или имя хоста
ssh_port	Integer	Порт SSH (по умолч. 22)
account_uuid	UUID	ID учётной записи SSH
working_dir	String	Рабочая директория на хосте
java_path	String	Путь к Java на хосте
mock_port_min	Integer	Мин. порт для заглушек
mock_port_max	Integer	Макс. порт для заглушек
description	String	Описание хоста

Параметр `java_path` введён в реестр хостов по причине неоднородности целевой инфраструктуры: в корпоративных средах переменная окружения `JAVA_HOME` нередко не определена или указывает на версию JRE, не совместимую с требованиями конкретной заглушки. Указание явного пути к исполняемому файлу Java устраняет неоднозначность и гарантирует запуск процесса с нужной версией JDK вне зависимости от конфигурации окружения на хосте.

Параметры `mock_port_min` и `mock_port_max` задают диапазон TCP-портов, в пределах которого система будет динамически выбирать свободный порт при запуске каждой новой заглушки на данном хосте. Ограничение диапазона необходимо для совместимости с политиками межсетевого экрана, которые в закрытых корпоративных сетях, как правило, разрешают входящий трафик только на заранее согласованные порты.

Поле `description` позволяет операторам документировать назначение каждого хоста непосредственно в интерфейсе системы, что упрощает эксплуатацию при работе с несколькими окружениями.

2.1.7 Диаграмма развёртывания

Диаграмма развёртывания (рисунок 4) описывает физическое размещение компонентов системы на серверной инфраструктуре. Сервер управляющего приложения содержит два постоянных компонента: FastAPI-процесс под управлением Uvicorn и экземпляр Redis. Бинарные файлы заглушек постоянно хранятся исключительно на целевых хостах в рабочих директориях. Временный каталог на сервере управляющего приложения используется лишь на время передачи файла (в течение одной SCP-операции) и очищается немедленно после её завершения.

Рисунок 4 – Диаграмма развёртывания

На целевых хостах не требуется установки каких-либо компонентов разрабатываемой системы. Обязательные условия для включения хоста в реестр: работающий SSH-сервер, настроенная учётная запись пользователя с правами на запись в рабочую директорию и наличие исполняемого файла Java по пути, указанному в параметре `java_path`. Бинарные файлы заглушек хранятся в рабочей директории хоста на протяжении всего времени существования записи о заглушке в системе и удаляются только при явном удалении заглушки оператором.

2.1.8 Сценарии взаимодействия компонентов

Для иллюстрации сквозного взаимодействия компонентов рассмотрим пять типовых сценариев. Принципиальное архитектурное решение: копирование бинарного артефакта на целевой хост выполняется однократно при регистрации заглушки; все последующие операции (запуск, остановка, переключение Rate Limiter) работают с файлом, уже находящимся на целевом хосте.

Сценарий 1. Регистрация новой заглушки.

а) Инженер тестирования загружает `.jar`- или `.war`-файл через веб-интерфейс и задаёт параметры: JVM-аргументы, целевой хост из реестра, пороговое значение Rate Limiter. Флаг `rate_limit_enabled` устанавливается в `false` по умолчанию.

б) REST API сохраняет файл во временный каталог на сервере управляющего приложения.

в) Lifecycle Manager получает параметры целевого хоста из Redis (адрес, учётная запись, рабочая директория). Crypto Service дешифрует пароль учётной записи.

г) Lifecycle Manager копирует файл с сервера управляющего приложения на целевой хост посредством SCP в рабочую директорию.

д) После успешного завершения SCP временный файл удаляется с сервера управляющего приложения.

е) В Redis создаётся Hash-запись заглушки с ключом `mocks:filename` командой `HSET`. Имя файла добавляется в реестр `mocks:registry` командой `SADD`. Обе операции выполняются в рамках одного конвейера Redis (Pipeline) с поддержкой транзакционности. Статус устанавливается в `REGISTERED`.

ж) Веб-интерфейс отображает новую заглушку на Dashboard со статусом «Зарегистрирована» и переключателем Rate Limiter в положении «Выкл.».

Сценарий 2. Запуск зарегистрированной заглушки.

а) Инженер тестирования нажимает кнопку «Запустить» для выбранной заглушки на Dashboard.

б) Браузер отправляет POST-запрос к эндпоинту `/api/v1/mocks/{mock_name}/start`.

в) Lifecycle Manager считывает из Redis конфигурацию заглушки (JVM-аргументы) и параметры хоста (адрес, `java_path`, учётная запись). Crypto Service дешифрует пароль.

г) Lifecycle Manager определяет свободный TCP-порт в диапазоне `mock_port_min / mock_port_max` через SSH-команду на целевом хосте.

д) Lifecycle Manager выполняет SSH-команду: `{java_path} {jvm_args} -jar {artifact_path} -server.port={port}`. Процесс запускается в фоновом режиме (`nohup`).

е) PID процесса, назначенный порт и статус `RUNNING` сохраняются в запись заглушки в Redis.

ж) Proxy Engine начинает маршрутизировать входящие запросы на зафиксированный порт заглушки.

Сценарий 3. Обработка входящего запроса через прокси.

а) Потребитель направляет HTTP-запрос на адрес вида `/ {mock_name} / {path}`.

б) Proxy Engine извлекает из Redis запись заглушки по `mock_name`: адрес, порт, статус, флаг `rate_limit_enabled`.

в) Если статус отличен от `RUNNING` – возвращается HTTP 503 (Service Unavailable).

г) Если `rate_limit_enabled = false` – запрос немедленно передаётся целевой заглушке, счётчики не обновляются. Если `rate_limit_enabled = true` – Rate Limiter инкрементирует счётчик `rate:{mock_name}:{window}` и сравнивает с порогом `rate_limit`. При превышении – возвращается HTTP 429 (Too Many Requests).

д) Proxy Engine асинхронно передаёт запрос целевой заглушке через `httpx` и возвращает ответ потребителю в неизменном виде.

Сценарий 4. Переключение Rate Limiter на Dashboard.

а) Оператор переключает тумблер Rate Limiter для выбранной заглушки на Dashboard.

б) Браузер отправляет PATCH-запрос к `/api/v1/mocks/{mock_name}/rate-limit` с телом `{"enabled": true/false}`.

в) REST API выполняет атомарную команду Redis HSET, записывая новое значение поля `rate_limit_enabled` («true» или «false») непосредственно в Hash-запись заглушки. Поскольку обновляется единственное поле одной командой, операция является атомарной и не требует дополнительных механизмов блокировки.

г) Начиная со следующего запроса к данной заглушке, Proxy Engine применяет обновлённое значение флага без перезапуска.

Сценарий 5. Удаление заглушки.

а) Инженер тестирования инициирует удаление заглушки через веб-интерфейс.

б) Если заглушка в состоянии RUNNING – Lifecycle Manager выполняет остановку процесса через SSH (SIGTERM, при необходимости SIGKILL).

в) Lifecycle Manager удаляет бинарный файл заглушки с целевого хоста командой `SSH rm {artifact_path}`.

г) Запись о заглушке и все связанные ключи (счётчики Rate Limiter) удаляются из Redis командами DEL и SREM.

д) Запись исчезает из Dashboard.

2.2 Проектирование структур данных для хранения конфигураций и состояний в Redis

2.2.1 Обоснование модели данных

Проектирование структур данных является фундаментальным этапом, определяющим производительность, корректность и сопровождаемость системы в целом. В разрабатываемом программном комплексе Redis выполняет роль единственного хранилища всех оперативных данных: конфигураций заглушек, параметров хостов и учётных записей, состояний запущенных процессов, счётчиков алгоритма Rate Limiter и вспомогательных индексов. Бинарные файлы артефактов в Redis не хранятся и не кэшируются – единственной постоянной копией каждого артефакта является файл на целевом хосте.

Выбор Redis в качестве хранилища обусловлен его архитектурными характеристиками, подробно рассмотренными в разделе 1.3: время доступа к любому ключу составляет менее 1 мс, что критически важно для алгоритма Rate Limiter, выполняемого при обработке каждого входящего запроса.

Redis предоставляет несколько базовых типов данных: строки (String), хеши (Hash), списки (List), множества (Set) и сортированные множества (Sorted Set). Для хранения конфигурационных объектов — заглушек, хостов, учётных записей и глобальных настроек — выбран тип Hash (HSET/HGETALL). Каждое поле объекта хранится как отдельное поле хеша, что позволяет обновлять конкретные атрибуты одной командой HSET без перезаписи всей записи. При необходимости прочитать объект целиком используется команда HGETALL; при чтении отдельного поля — HGET; при пакетном обновлении нескольких полей одновременно — HSET с передачей словаря (mapping). Для хранения реестров (множеств идентификаторов) используется тип Set, гарантирующий уникальность элементов. Для счётчиков Rate Limiter применяется тип String со встроенными атомарными командами INCR и EXPIRE.

2.2.2 Структура данных заглушки

Центральным объектом системы является запись о заглушке — имитационном Java-сервисе, управляемом программным комплексом. Запись хранится по ключу вида `mocks:{filename}`, где `{filename}` — имя загруженного файла артефакта (например, `payment-service-stub.jar`). Имя файла является уникальным идентификатором заглушки в системе: при попытке зарегистрировать файл с именем, уже присутствующим в реестре `mocks:registry`, система возвращает ошибку и предлагает оператору переименовать загружаемый артефакт или удалить существующую запись.

Запись заглушки хранит полный путь к бинарному файлу в поле `artifact_path`. Значение вычисляется однократно при регистрации как

конкатенация поля `working_dir` из записи хоста и имени файла артефакта, после чего сохраняется в Hash. Это исключает необходимость повторного обращения к записи хоста при каждой операции запуска или остановки.

Структура Hash-записи заглушки описана в листинге 2.1.

Листинг 2.1 – Структура Hash-записи заглушки в Redis

```
1 HSET mocks:{filename}
2   hostname          test-server-01
3   port              8105
4   pid               24731
5   status            RUNNING
6   jvm_args          -Xms128m -Xmx256m
7   rate_limit        500
8   rate_limit_enabled true
9   registered_at      2024-11-01T10:30:00+00:00
10  started_at        2024-11-01T10:35:00+00:00
11  artifact_path      /opt/mock-services/payment-service-stub.jar
```

Ключ: `mocks:{filename}`, тип: Hash. Все значения хранятся как строки; числовые поля (`port`, `pid`, `rate_limit`) приводятся к целому типу при чтении.

Описание полей записи заглушки приведено в таблице 4.

Таблица 4 – Поля Hash-записи заглушки

Поле	Тип	Описание
hostname	String	Хост, где запущена заглушка
port	Integer/null	TCP-порт процесса
pid	Integer/null	ID процесса на хосте
status	String	REGISTERED / RUNNING / STOPPED / ERROR
jvm_args	String	Аргументы JVM
rate_limit	Integer	Макс. запросов в секунду
rate_limit_enabled	Boolean	Флаг активности Rate Limiter
registered_at	ISO 8601	Дата регистрации

Поле	Тип	Описание
<code>started_at</code>	ISO 8601/null	Дата последнего запуска
<code>artifact_path</code>	String	Абсолютный путь к JAR/WAR на целевом хосте

Поля `port`, `pid` и `started_at` заполняются только после успешного запуска процесса и сбрасываются в пустую строку при его остановке или удалении (командой `HSET` с пустым строковым значением). Поле `artifact_path` содержит готовый абсолютный путь к файлу артефакта на целевом хосте и используется Lifecycle Manager напрямую при выполнении команд запуска и удаления.

2.2.3 Структура данных хоста

Запись о целевом хосте хранится по ключу вида `hosts:{hostname}`, где `hostname` — сетевое имя или IP-адрес хоста. `Hostname` одновременно выполняет роль уникального идентификатора записи и адреса для SSH/SCP-подключения. Прямой доступ к конфигурации хоста по известному `hostname` осуществляется за одну операцию `HGETALL` без дополнительного индекса.

Структура Hash-записи хоста описана в листинге 2.2.

Листинг 2.2 – Структура Hash-записи хоста в Redis

```

1 HSET hosts:{hostname}
2  ssh_port          22
3  account_uuid      e5f6a7b8-c9d0-1234-efab-567890abcdef
4  working_dir        /opt/mock-services
5  java_path          /usr/lib/jvm/java-17-openjdk-amd64/bin/java
6  mock_port_min      8100
7  mock_port_max      8200
8  description        Стенд нагрузочного тестирования – контур препром.
9  status             AVAILABLE
10 last_checked_at    2024-11-01T10:29:55+00:00

```

Ключ: `hosts:{hostname}`, тип: Hash. Числовые поля (`ssh_port`, `mock_port_min`, `mock_port_max`) приводятся к целому типу при чтении.

Поле `status` отражает результат последней проверки доступности SSH-соединения, выполняемой фоновой задачей с настраиваемым интервалом. Допустимые значения: `AVAILABLE` (соединение успешно установлено), `UNAVAILABLE` (попытка соединения завершилась ошибкой) и `UNKNOWN` (проверка ещё не выполнялась). Поле `last_checked_at` фиксирует время последней проверки для отображения актуальности статуса в веб-интерфейсе.

Поле `account_uuid` содержит UUID учётной записи SSH из реестра `accounts`, используемой для подключения к данному хосту. Lifecycle Manager читает это поле командой `HGET` при каждой SSH/SCP-операции для получения соответствующей записи `accounts:{uuid}` с зашифрованным паролем.

2.2.4 Структура данных учётной записи

Запись об учётной записи ОС Linux хранится по ключу вида `accounts:{uuid}`. Учётная запись содержит данные, необходимые для аутентификации при SSH/SCP-операциях на целевых хостах. Пароль хранится в зашифрованном виде – компонент `Crypto Service` шифрует его алгоритмом AES-128 (Fernet) перед записью в Redis. Ключ шифрования хранится на файловой системе сервера в файле с правами 600.

Структура Hash-записи учётной записи описана в листинге 2.3.

Листинг 2.3 – Структура Hash-записи учётной записи в Redis

```
1 HSET accounts:{uuid}
2     username      mock-runner
3     password_enc   gAAAAAB1...base64-encoded-ciphertext...
4     description    Сервисный пользователь для запуска заглушек
5     created_at      2024-10-15T09:00:00+00:00
```

Ключ: `accounts:{uuid}`, тип: Hash.

Поле `password_enc` содержит Base64-кодированный зашифрованный блок, полученный библиотекой `cryptography` (Fernet). При необходимости установить SSH-соединение `Crypto Service` читает это поле командой `HGET` и выполняет дешифрование в оперативной памяти процесса; расшифрованный пароль никогда не записывается на диск и не сохраняется в `Redis`. Такой подход обеспечивает защиту паролей в случае несанкционированного получения дампа данных `Redis`.

2.2.5 Структуры данных реестров

Для хранения множеств идентификаторов всех зарегистрированных объектов используется тип `Set`. Каждый реестр хранится под фиксированным ключом и содержит строковые идентификаторы соответствующих объектов.

Реестр заглушек (`mocks:registry`) – множество имён файлов всех зарегистрированных заглушек. При регистрации новой заглушки её имя добавляется командой `SADD`, при удалении – удаляется командой `SREM`. Команда `SMEMBERS` возвращает полный список для формирования `Dashboard`.

Реестр хостов (`hosts:registry`) – множество `hostname`-идентификаторов зарегистрированных целевых хостов (сетевое имя или IP-адрес). При регистрации нового хоста его `hostname` добавляется командой `SADD`, при удалении – командой `SREM`. Команда `SMEMBERS` возвращает полный список для отображения в интерфейсе настроек.

Реестр учётных записей (`accounts:registry`) – множество `UUID` зарегистрированных учётных записей `SSH`.

Применение типа `Set` гарантирует уникальность идентификаторов: повторное добавление уже существующего элемента командой `SADD` не создаёт дубликата и возвращает `0`, что используется системой для обнаружения попытки регистрации заглушки с дублирующим именем файла.

ER-диаграмма приведена на рисунке 5.



Рисунок 5 – ER-диаграмма объектов в Redis

2.2.6 Структура данных счётчика Rate Limiter

Алгоритм Rate Limiter реализован на основе паттерна «Счётчик с фиксированным временным окном» (Fixed Window Counter). Для каждой заглушки и каждого временного окна в Redis хранится отдельный ключ-счётчик. Структура ключа: `rate:{filename}:{window}`, где `{window}` – метка временного окна, вычисляемая как целочисленное деление текущего Unix-timestamp на длину окна в секундах.

Например, при длине окна 1 секунда и текущем времени 1700000060.7 метка окна равна 1700000060, а ключ для заглушки `payment-service-stub.jar` примет вид `rate:payment-service-stub.jar:1700000060`. Все запросы, поступившие в течение одной секунды, инкрементируют один и тот же счётчик.

Алгоритм обработки каждого запроса через Rate Limiter описан в листинге 2.4.

Листинг 2.4 – Псевдокод алгоритма Fixed Window Counter

```
1 def check_rate_limit(filename: str, limit: int, window_size: int = 1)
   ↪ -> bool:
2     # Шаг 1: вычислить метку текущего окна
3     window = floor(current_unix_timestamp() / window_size)
4     key      = f"rate:{filename}:{window}"

5     # Шаг 2: атомарный инкремент счётчика
6     count = INCR(key)

7     # Шаг 3: установить TTL при создании нового окна
8     if count == 1:
9         EXPIRE(key, window_size * 2) # TTL = 2 окна (запас на сдвиг
   ↪ времени)

10    # Шаг 4: проверить лимит
11    if count > limit:
12        return False # запрос отклонён -> HTTP 429
13    return True      # запрос разрешён
```

Установка TTL через команду EXPIRE выполняется только при создании нового счётчика (когда INCR возвращает 1), то есть при первом запросе в новом временном окне. TTL устанавливается равным удвоенному размеру окна для гарантированного самоочищения устаревших счётчиков даже при незначительных расхождениях системного времени между клиентом и сервером. Это исключает накопление «мёртвых» ключей в Redis и предотвращает утечку памяти при длительной эксплуатации системы.

Атомарность операции INCR [25] является ключевым свойством данного подхода: Redis выполняет её как единую неделимую операцию, что гарантирует корректный подсчёт при конкурентном поступлении тысяч запросов в рамках одного временного окна без использования транзакций или блокировок.

2.2.7 Глобальные настройки системы

Глобальные конфигурационные параметры приложения, не привязанные к конкретному хосту или заглушке, хранятся под ключом `settings:global`. Запись содержит параметры, задаваемые при первоначальной настройке системы и редко изменяемые в процессе эксплуатации.

Структура Hash-записи глобальных настроек описана в листинге 2.5.

Листинг 2.5 – Структура Hash-записи глобальных настроек в Redis

```
1 HSET settings:global
2     rate_limit_window_size      1
3     host_check_interval_sec     30
4     proxy_timeout_sec           10
5     log_retention_lines         1000
```

Ключ: `settings:global`, тип: Hash. Значения хранятся как строки; числовые поля приводятся к целому типу при чтении.

Описание полей глобальных настроек приведено в таблице 5.

Таблица 5 – Поля Hash-записи глобальных настроек системы

Поле	Тип	Описание
<code>rate_limit_window_size</code>	Integer	Размер окна Rate Limiter (сек)
<code>host_check_interval_sec</code>	Integer	Интервал проверки хостов (сек)
<code>proxy_timeout_sec</code>	Integer	Таймаут проксирования (сек)
<code>log_retention_lines</code>	Integer	Макс. строк лога в буфере

2.2.8 Персистентность данных и стратегия восстановления

Redis настроен в режиме AOF (Append Only File)[9] с параметром `appendfsync everysec`: каждая операция записи фиксируется в журнале, синхронизируемом с диском раз в секунду. Максимальная потеря данных при аварии составляет не более одной секунды.

Стратегия восстановления после перезапуска: управляющее приложение читает реестр заглушек из Redis командой SMEMBERS, затем для каждой заглушки считывает все поля Hash командой HGETALL. Для каждой заглушки со статусом RUNNING выполняется проверка фактического наличия процесса на целевом хосте. Если процесс обнаружен — статус подтверждается и восстанавливается HTTP-клиент прокси-движка с прежними параметрами. Если процесс не обнаружен, либо значения полей pid и port отсутствуют или повреждены, — система автоматически предпринимает попытку перезапуска заглушки: выполняется поиск свободного TCP-порта на целевом хосте и повторный запуск Java-процесса с сохранёнными JVM-аргументами. В случае успешного перезапуска запись в Redis обновляется новыми значениями pid, port и started_at. В случае неудачи ошибка фиксируется в журнале приложения, а конфигурация заглушки в Redis сохраняется в неизменном виде для последующего ручного вмешательства оператора.

В таблице 6 представлена сводная матрица операций чтения и записи Redis по компонентам системы.

Таблица 6 – Матрица операций Redis по компонентам системы

Компонент	Операции Redis	Назначение
Lifecycle Manager	HSET, HGETALL, HGET, SADD, SREM, DEL	Управление процессами
Proxy Engine	HGETALL, HMGET	Чтение данных заглушки
Rate Limiter	INCR, EXPIRE	Счётчик запросов
Settings Module	HSET, HGETALL, HGET, SADD, SREM, DEL	Управление конфигами
Metrics Exporter	SMEMBERS, HGETALL (pipeline)	Сбор метрик

Для операции Metrics Exporter применяется механизм конвейеризации команд Redis (Pipeline): сначала читается полный список ключей из реестра, затем все операции чтения конфигураций отправляются единым пакетом.

Это снижает суммарные сетевые задержки при агрегации данных о большом числе заглушек с $O(n)$ до фактически $O(1)$ по числу сетевых round-trip.

2.3 Разработка макетов приложения

Проектирование пользовательского интерфейса является самостоятельным этапом разработки программного комплекса. Высокодетализированные прототипы экранных форм позволяют верифицировать логику навигации, выявить противоречия в расположении элементов управления и зафиксировать визуальное соглашение до начала реализации HTML-шаблонов. В соответствии с методическими требованиями в данном разделе обосновывается выбор цветового решения, типографики и стиля интерфейса, а также приводятся прототипы всех ключевых экранных форм разрабатываемой системы.

2.3.1 Обоснование стиля и цветового решения

Целевая аудитория системы – инженеры тестирования, работающие в различных условиях. Эксплуатация системы носит интенсивный характер: оператор одновременно отслеживает состояние нескольких десятков заглушек, переключает тумблеры Rate Limiter, запускает и останавливает процессы. Это определяет два ключевых требования к интерфейсу: минимальную утомляемость при длительной работе и мгновенную читаемость статусов без необходимости анализировать текстовые подписи.

Для выполнения первого требования выбрана тёмная цветовая схема (dark theme): светлый текст на тёмном фоне снижает нагрузку на зрение при многочасовой работе и является де-факто стандартом для инструментов класса DevOps и Operations. Для выполнения второго требования разработана система цветовых статусных индикаторов, в которой цвет однозначно кодирует состояние объекта: зелёный соответствует активному состоянию (RUNNING),

жёлтый – промежуточному (REGISTERED / STOPPED), красный – ошибочному (ERROR / UNAVAILABLE).

Акцентный цвет интерфейса – синий (#4F8EF7). Данный оттенок обеспечивает достаточный контраст по отношению к тёмному фону (коэффициент контрастности превышает 4,5:1, что соответствует требованиям WCAG 2.1 уровня AA[26]) и не несёт эмоциональной нагрузки, характерной для статусных цветов. Акцентным цветом выделяются интерактивные элементы: кнопки основного действия, активные пункты навигации и ссылки.

Типографическую основу интерфейса составляет шрифт Inter, который входит в базовую поставку большинства дистрибутивов Linux и не требует загрузки с внешних CDN. Это соответствует требованию технологической независимости от зарубежных сервисов, сформулированному в разделе 1.3. Основной размер шрифта – 14 px, межстрочный интервал – 1.5. Для отображения технических идентификаторов (имена файлов, JVM-аргументы, пути к директориям) используется моноширинный шрифт JetBrains Mono, обеспечивающий лучшую читаемость длинных строк технического содержания.

Компоновка страниц построена на двухколоночной сетке: постоянная боковая панель навигации шириной 220 px и основная рабочая область, занимающая оставшееся пространство. Такая компоновка является стандартной для административных веб-панелей и обеспечивает предсказуемость навигации при переключении между разделами системы. Сводная характеристика визуальной системы интерфейса приведена в таблице 7.

Таблица 7 – Визуальная система интерфейса

Элемент	Значение	Назначение
Фон контента	#121218	Основной фон
Фон боковой панели	#1A1A24	Навигация
Акцентный цвет	#4F8EF7	Кнопки, ссылки
Статус RUNNING	#22C55E	Процесс запущен

Элемент	Значение	Назначение
Статус REGISTERED / STOPPED	#EAB308	Зарегистрирован, не запущен
Статус ERROR / UNAVAILABLE	#EF4444	Ошибка/недоступен
Основной текст	#E2E8F0	Основной контент
Вторичный текст	#94A3B8	Метки, подсказки
Основной шрифт	Inter	Текст
Моноширинный шрифт	JetBrains Mono	Код, пути, аргументы

2.3.2 Структура навигации и общий хром приложения

Все экранные формы системы разделяют единый шаблон-«обёртку», включающий постоянную боковую панель навигации и верхнюю строку с заголовком текущего раздела. Структура навигации включает три основных раздела:

- **Dashboard** – главная панель управления заглушками; отображается по умолчанию при входе в систему;
- **Настройки** – управление хостами и учётными записями SSH, глобальные параметры системы;
- **Журналы** – просмотр вывода Java-процессов заглушек.

Боковая панель содержит логотип системы «MockControl» в верхней части и строку версии в нижней. Правый верхний угол заголовочной полосы показывает оперативный статус подключения к Redis, что позволяет оператору немедленно обнаруживать сбои хранилища, не выполняя диагностических действий. Макет общего хрома приложения с боковой навигацией представлен на рисунке 6.

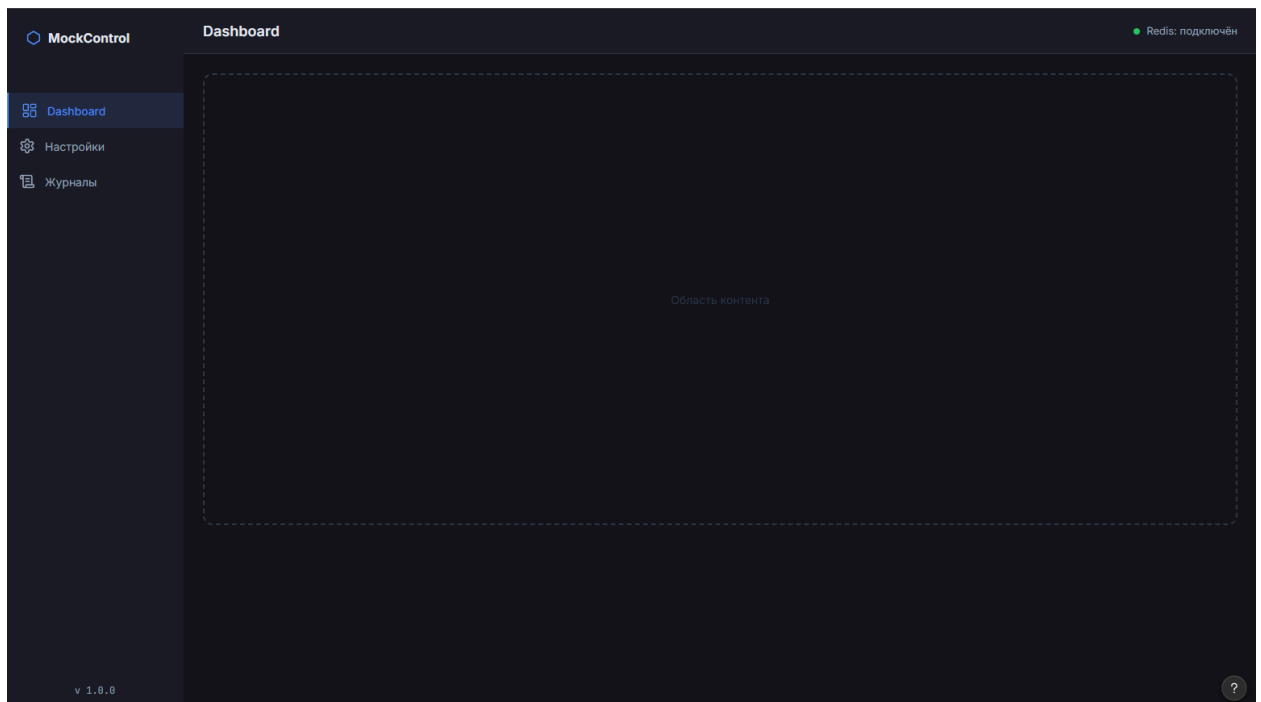


Рисунок 6 – Макет общего хрома приложения с боковой навигацией

2.3.3 Макет главной панели управления (Dashboard)

Dashboard является основным рабочим экраном системы. Он отображает полный список зарегистрированных заглушек и предоставляет оператору возможность управлять их жизненным циклом непосредственно из таблицы без перехода на отдельные страницы.

Верхняя часть рабочей области занята четырьмя информационными карточками со сводной статистикой: общее число зарегистрированных заглушек, число активных (RUNNING), число заглушек в статусе ошибки и число доступных хостов. Карточки позволяют оценить состояние всей инфраструктуры одним взглядом без анализа содержимого таблицы.

Таблица заглушек содержит столбцы: имя файла (уникальный идентификатор, отображается моноширинным шрифтом), целевой хост, статус процесса в виде цветного бейджа, назначенный порт, тумблер Rate Limiter и кнопки действий (Запустить / Остановить / Удалить). Кнопка «Зарегистрировать заглушку» в правом верхнем углу открывает модальное окно загрузки артефакта.

Макет Dashboard с заполненной таблицей представлен на рисунке 7.

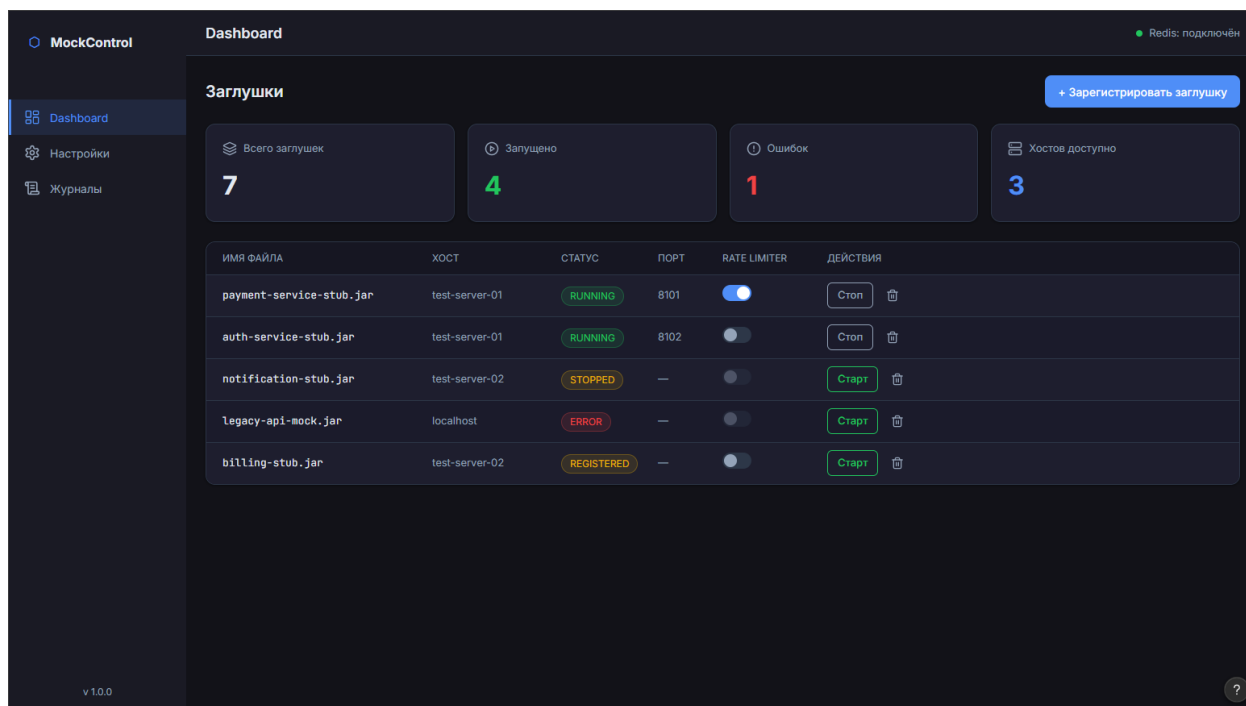


Рисунок 7 – Макет главной панели управления (Dashboard)

2.3.4 Макет модального окна регистрации заглушки

Регистрация новой заглушки выполняется через модальное окно, открывающееся поверх Dashboard без перехода на отдельную страницу. Такой подход сохраняет контекст: оператор видит уже зарегистрированные заглушки за полупрозрачным затемнённым фоном и может убедиться в отсутствии конфликта имён до подтверждения операции.

Форма содержит пять полей: зону перетаскивания файла артефакта (.jar / .war), выпадающий список целевого хоста, текстовое поле JVM-аргументов, числовое поле лимита запросов Rate Limiter и флажок немедленного запуска после регистрации. Поле JVM-аргументов отображается моноширинным шрифтом, что снижает вероятность ошибок при вводе длинных строк аргументов.

Макет Dashboard с открытым модальным окном регистрации представлен на рисунке 8.

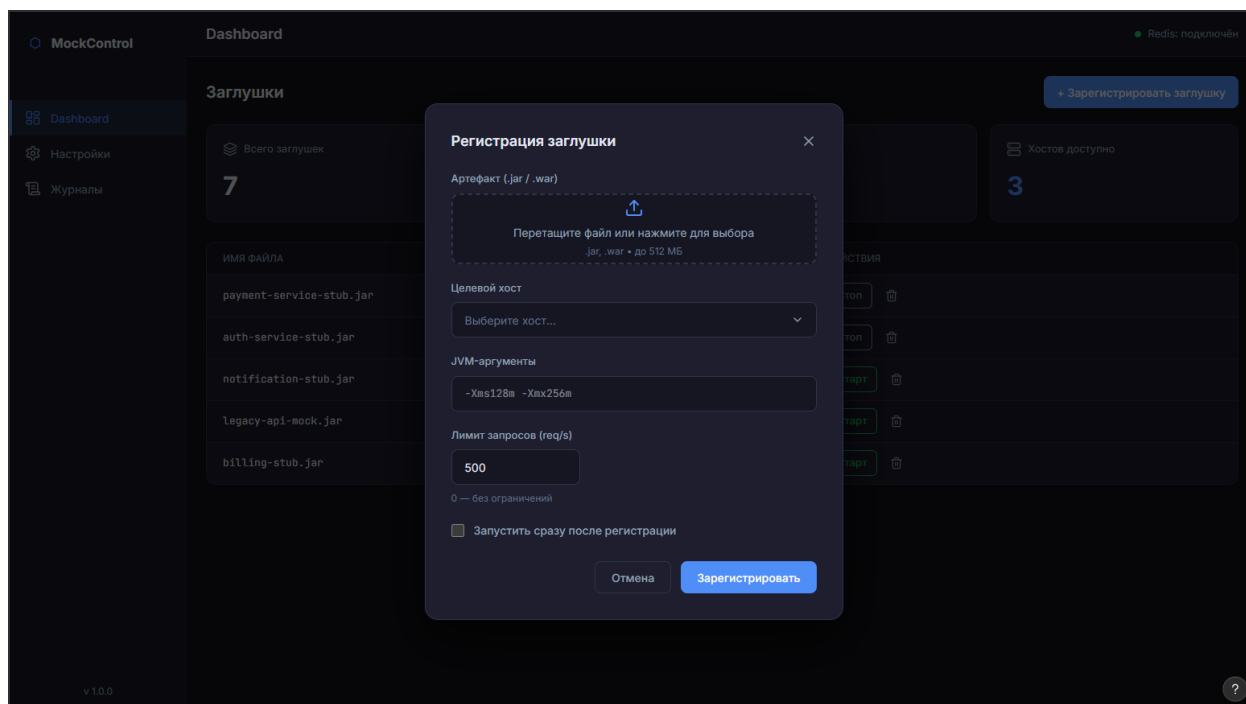


Рисунок 8 – Макет модального окна регистрации новой заглушки

2.3.5 Макет страницы настроек: управление хостами

Страница настроек организована в виде двух горизонтальных вкладок: «Хосты» и «Учётные записи». Такое разделение предотвращает перегруженность экрана и группирует логически связанные объекты.

Вкладка «Хосты» отображает таблицу зарегистрированных целевых серверов с индикатором доступности SSH-соединения для каждого хоста. Столбец «Проверено» показывает дату и время последней проверки доступности, что позволяет оценить актуальность отображаемого статуса. Кнопка «Добавить хост» открывает модальную форму с полями, описанными в разделе 2.1.6.

Вкладка «Учётные записи» отображает список сохранённых учётных записей ОС Linux для SSH/SCP-операций. Пароли в таблице не отображаются: вместо них показывается замаскированная строка «●●●●●●●●», что исключает случайную утечку при демонстрации интерфейса коллегам или на презентации.

Макет страницы настроек (вкладка «Хосты») представлен на рисунке 9, форма добавления нового хоста – на рисунке 10.

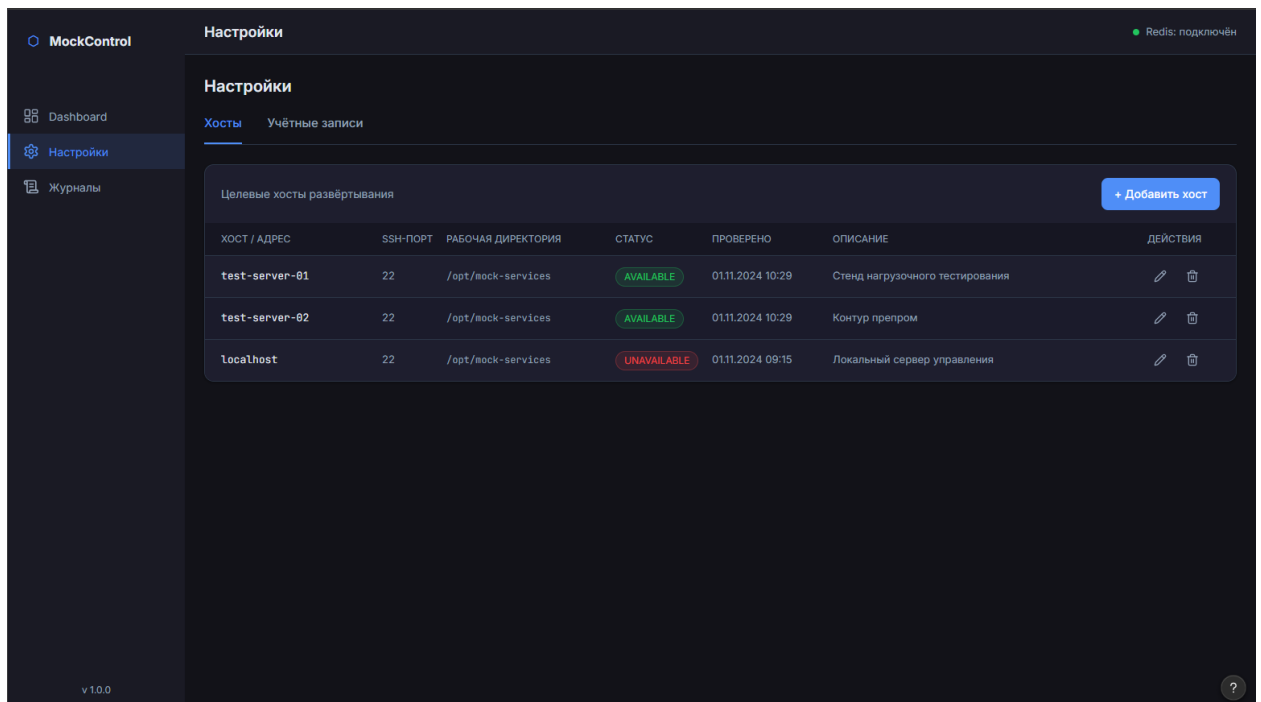


Рисунок 9 – Макет страницы настроек, вкладка «Хосты»

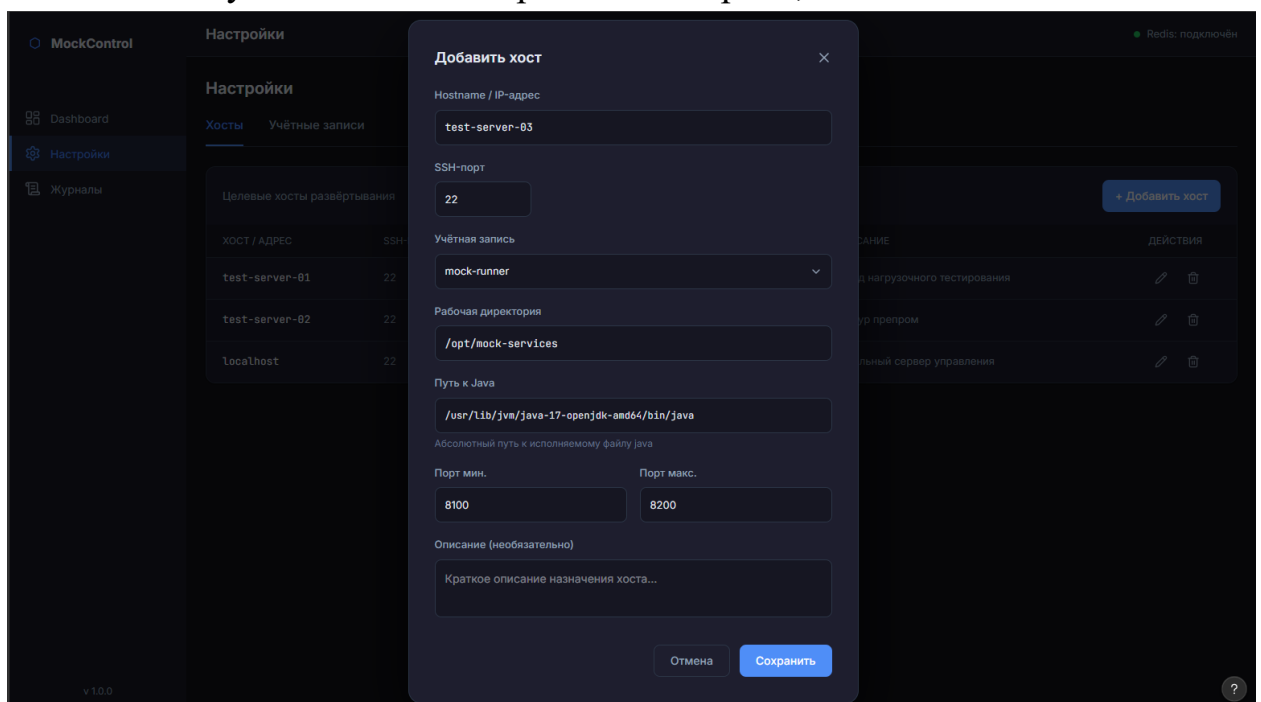


Рисунок 10 – Макет модального окна добавления нового хоста

2.3.6 Макет страницы просмотра журналов процессов

Страница журналов предназначена для просмотра стандартного вывода (stdout/stderr) Java-процессов заглушек. Выбор заглушки осуществляется через выпадающий список, что позволяет переключаться между логами без перехода на отдельные страницы.

Основную часть экрана занимает терминальный блок с монохромным текстом на чёрном фоне. Строки, содержащие ключевые слова **ERROR** и **WARN**, выделяются соответственно красным и жёлтым фоном для быстрого визуального поиска проблем. Метки временных меток окрашены в нейтральный серо-синий цвет, чтобы не конкурировать с читаемостью основного текста.

Панель управления над терминальным блоком содержит: выпадающий список заглушек, тумблер автообновления, кнопку ручного обновления и кнопку очистки отображаемого буфера. Строка состояния между панелью управления и терминальным блоком показывает PID процесса и назначенный порт, что позволяет оператору проверить корректность отображаемых данных. Макет страницы журналов с заполненным выводом представлен на рисунке 11.

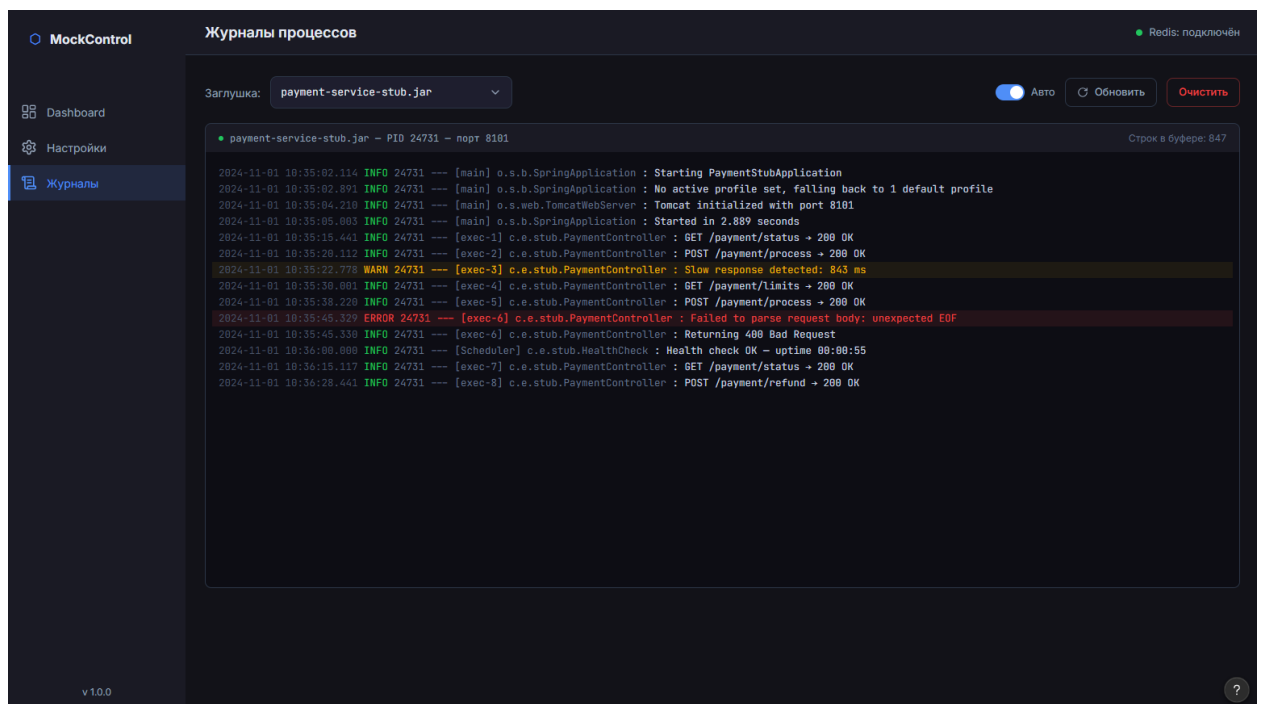


Рисунок 11 – Макет страницы просмотра журналов процессов

2.3.7 Итоги проектирования интерфейса

В рамках данного раздела разработаны высокодетализированные прототипы шести ключевых экранных форм программного комплекса: общий хром приложения с навигацией (рисунок 6), главная панель управления Dashboard (рисунок 7), модальное окно регистрации заглушки (рисунок 8),

страница настроек с таблицей хостов (рисунок 9), форма добавления нового хоста (рисунок 10) и страница журналов процессов (рисунок 11).

Все экранные формы выдержаны в единой визуальной системе с тёмной цветовой схемой, акцентным синим цветом (#4F8EF7) и цветовым кодированием статусов объектов через бейджи и индикаторы. Цветовая система обеспечивает мгновенную оценку состояния инфраструктуры без чтения текстовых подписей.

Разработанные прототипы обосновывают соответствие интерфейса современным требованиям к инструментам класса Operations: минимум навигационных переходов для выполнения целевых задач, низкий порог освоения для инженеров тестирования без специальной веб-разработческой компетенции, а также технологическая независимость от внешних шрифтовых и ресурсных CDN.

2.4 Реализация асинхронных механизмов управления процессами и ввода пользовательских параметров.

2.4.1 Архитектура асинхронного управления процессами

Ключевым требованием к модулю управления жизненным циклом является обеспечение неблокирующей модели выполнения: операции запуска, остановки и мониторинга Java-процессов не должны препятствовать обработке входящих HTTP-запросов в прокси-движке и веб-интерфейсе. Все методы класса `LifecycleManager` реализованы как корутины языка Python с использованием ключевых слов `async/await`, что позволяет событийному циклу `asyncio`[3] переключаться между операциями ввода-вывода без блокировки вычислительного потока.

Операции управления процессами делятся на две категории в зависимости от типа целевого хоста, определяемого функцией `host_resolver.is_local(hostname)`: для локального хоста применяются встроенные средства Python и системные вызовы операционной

системы, тогда как для удалённого хоста все действия выполняются через пул постоянных SSH-соединений (`SSHPool`), реализованный на базе библиотеки `asyncssh`. Такое ветвление обеспечивает единый программный интерфейс при принципиально различных транспортных механизмах.

Функция `is_local` использует `functools.lru_cache` для кэширования результатов по значению аргумента: при первом вызове функция собирает полное множество сетевых идентификаторов локального хоста – адреса `127.0.0.1` и `::1`, результаты `socket.gethostbyname_ex`, `socket.getaddrinfo`, а также IP-адрес, определённый через UDP-зонд к известному публичному адресу, – и все последующие вызовы возвращают кэшированный результат без повторного обращения к сетевому стеку.

2.4.2 Запуск Java-процесса на локальном хосте

При запуске заглушки на локальном хосте формирование и исполнение команды производятся через `asyncio.create_subprocess_exec`, использование которой вместо `subprocess.Popen` принципиально важно: она интегрируется непосредственно в событийный цикл и не блокирует его на время создания дочернего процесса.

Перед вызовом `create_subprocess_exec` выполняется поиск свободного TCP-порта в указанном диапазоне (`mock_port_min` – `mock_port_max`). Поскольку проверка доступности порта требует системного вызова `socket.bind`, который является блокирующим, она вынесена в `asyncio.to_thread` – функцию, запускающую синхронный код в пуле потоков без блокировки основного событийного цикла.

Реализация запуска Java-процесса на локальном хосте приведена в листинге 2.6.

Листинг 2.6 – Запуск Java-процесса на локальном хосте

```
1 async def start_mock(self, filename: str) -> None:
2     # ... загрузка конфигурации из Redis ...
3     if self._is_local(hostname):
4         # Поиск свободного порта в пуле потоков (блокирующий вызов)
5         port = await self._pick_free_port_local(pmin, pmax)
6         log_file_path = _log_path_for_filename(filename)
7
8         # Открытие лог-файла вынесено в поток (I/O-bound операция)
9         lf = await asyncio.to_thread(open, log_file_path, "ab")
10        try:
11            # Разбор JVM-аргументов из строки в список токенов
12            argv = [java_path]
13            argv.extend(shlex.split(jvm_args) if jvm_args.strip()
14                        ↪ else [])
15            argv.extend(["-jar", artifact_path, f"{port}"])
16
17            # Неблокирующий запуск дочернего процесса
18            proc = await asyncio.create_subprocess_exec(
19                *argv,
20                stdin=asyncio.subprocess.DEVNULL,
21                stdout=lf,
22                stderr=asyncio.subprocess.STDOUT,
23                start_new_session=True,    # изоляция от группы
24                ↪ процессов
25            )
26        finally:
27            lf.close()
28        pid = proc.pid
```

Параметр `start_new_session=True` создаёт новый сеанс операционной системы для дочернего процесса: Java-процесс не входит в группу процессов управляющего приложения и продолжит работу при отправке сигнала `SIGTERM` родительскому процессу, что критично для сценариев перезапуска управляющего сервиса.

Стандартный поток вывода и поток ошибок перенаправляются в файловый дескриптор журнала (`stdout=1f`, `stderr=asyncio.subprocess.STDOUT`), стандартный поток ввода явно закрывается (`stdin=DEVNULL`), что предотвращает зависание Java-приложений, ожидающих ввода при интерактивном запуске.

2.4.3 Обработка JVM-аргументов пользователя

JVM-аргументы сохраняются в Redis в виде единой строки – поле `jvm_args` записи заглушки – и могут содержать произвольные параметры виртуальной машины Java, например: `-Xms128m -Xmx512m -Dspring.profiles.active=test`.

Для корректного разбора этой строки в список токенов, совместимый с сигнатурой `create_subprocess_exec`, применяется функция `shlex.split`, которая выполняет лексический разбор строки по правилам POSIX-оболочки: корректно обрабатывает кавычки, экранирование специальных символов и пробельные разделители. Применение `shlex.split` вместо наивного разбиения по пробелу (`str.split`) позволяет пользователю задавать аргументы со встроенными пробелами в кавычках, что необходимо для параметров вида `Dmy.property=value with spaces`.

Пример трансформации строки JVM-аргументов в аргументный вектор процесса приведён в листинге 2.7.

Листинг 2.7 – Разбор строки JVM-аргументов в аргументный вектор

```
1 # Входная строка из Redis (поле jvm_args)
2 jvm_args = "-Xms128m -Xmx512m -Dspring.profiles.active=test"

3 # Формирование аргументного вектора
4 argv = ["/usr/lib/jvm/java-17-openjdk-amd64/bin/java"]
5 argv.extend(shlex.split(jvm_args)) # -> ["-Xms128m", "-Xmx512m",
    ↪ ...]
6 argv.extend(["-jar", "/opt/mock-services/stub.jar", "8105"])

7 # Итоговый вектор, передаваемый в create_subprocess_exec:
8 # ["/usr/.../java", "-Xms128m", "-Xmx512m",
9 # "-Dspring.profiles.active=test", "-jar", "/opt/.../stub.jar",
    ↪ "8105"]
```

Аналогичная логика применяется при запуске на удалённом хосте, однако в этом случае токены дополнительно экранируются функцией `shlex.quote` перед включением в строку SSH-команды, что предотвращает инъекцию оболочки через специальные символы в пользовательских аргументах.

2.4.4 Запуск Java-процесса на удалённом хосте

При запуске заглушки на удалённом хосте управляющее приложение взаимодействует с хостом через `SSHPool` – объект, поддерживающий пул постоянных асинхронных SSH-соединений на базе библиотеки `asyncssh`[20]. Для каждого хоста поддерживается единственное соединение, защищённое `asyncio.Lock` от конкурентных операций при переподключении.

Перед запуском процесса на удалённом хосте выполняется поиск свободного TCP-порта: по SSH-соединению отправляется команда `ss -tln` (или `netstat -tln` в качестве запасного варианта), её вывод разбирается для получения множества занятых портов, и первый порт из заданного диапазона, не вошедший в это множество, используется для запуска заглушки.

Формирование SSH-команды запуска Java-процесса приведено в листинге 2.8.

Листинг 2.8 – Формирование SSH-команды запуска Java-процесса на удалённом хосте

```
1 # Экранирование JVM-аргументов для безопасной передачи через SSH
2 jvm_part = "".join(
3     shlex.quote(t) for t in shlex.split(jvm_args.strip())
4 ) if jvm_args.strip() else ""
5
6 # Итоговая команда: nohup, перенаправление вывода, вывод PID
7 cmd = (
8     f"nohup {shlex.quote(java_path)} {mid}"
9     f"-jar {shlex.quote(artifact_path)} {int(port)} "
10    f">> {shlex.quote(log_remote)} 2>&1 & echo $!"
11 )
12 proc = await self._ssh.run_command(
13     hostname, user, pwd, cmd, port=ssh_port, timeout=60.0
14 )
15 # Последняя строка вывода содержит PID запущенного процесса
16 pid = int(proc.stdout.strip().splitlines()[-1])
```

Использование `nohup` гарантирует, что Java-процесс продолжит работу после закрытия SSH-сессии, символ `&` немедленно переводит процесс в фоновый режим, а конструкция `echo $!` выводит PID запущенного фонового процесса в стандартный вывод, который управляющее приложение считывает как последнюю строку ответа и сохраняет в Redis для последующих операций остановки и контроля живости.

2.4.5 Остановка процесса и проверка его живости

Операция остановки реализует двухэтапный протокол завершения, унифицированный для локального и удалённого случаев. На первом этапе процессу отправляется сигнал `SIGTERM`, дающий приложению возможность

корректно завершить работу (graceful shutdown). На втором этапе в течение пяти секунд производится периодическая проверка живости процесса (50 итераций с интервалом 100 мс); если по истечении этого времени процесс ещё работает, отправляется сигнал `SIGKILL` для принудительного завершения. Ожидание реализовано через `asyncio.sleep(0.1)` внутри цикла, что позволяет событийному циклу обрабатывать другие запросы во время ожидания, не блокируя сервер.

Проверка живости процесса также ветвится по типу хоста. Для локального хоста используется системный вызов `os.kill(pid, 0)`, который не отправляет реального сигнала, а лишь проверяет существование процесса с указанным PID; этот вызов выполняется в `asyncio.to_thread` как блокирующий. Для удалённого хоста по SSH-соединению отправляется семантически эквивалентная команда `kill -0 {pid}`; ответ OK подтверждает, что процесс жив. Реализация обоих методов приведена в листинге 2.9.

Листинг 2.9 – Методы проверки живости процесса

```
1 # Локальная проверка: системный вызов в пуле потоков
2 async def _process_alive_local(self, pid: int) -> bool:
3     def check() -> bool:
4         try:
5             os.kill(pid, 0) # сигнал 0: только проверка существования
6         except OSError:
7             return False # процесс не найден или нет прав
8         return True
9
10    return await asyncio.to_thread(check)
11
12 # Удалённая проверка: команда kill -0 через SSH
13 async def _process_alive_remote(
14     self, hostname: str, username: str, password: str,
15     ssh_port: int, pid: int,
16 ) -> bool:
17     proc = await self._ssh.run_command(
18         hostname, username, password,
19         f"kill -0 {int(pid)} 2>/dev/null && echo OK || echo NO",
20         port=ssh_port, timeout=15.0,
21     )
22     return (proc.stdout or "").strip().endswith("OK")
```

2.4.6 Пул SSH-соединений

Для минимизации задержки при выполнении операций управления на удалённых хостах реализован пул постоянных SSH-соединений (SSHPool): установка SSH-соединения является дорогостоящей операцией (согласование ключей, аутентификация), тогда как повторное использование уже установленного соединения добавляет лишь сетевую задержку канала.

Для каждого хоста пул хранит одну запись (`_HostEntry`), содержащую `asyncio.Lock` против гонок при конкурентном переподключении и ссылку на соединение `SSHClientConnection`. Перед каждым ис-

пользованием соединение проверяется командой `true` с таймаутом 5 секунд; при неуспехе соединение пересоздаётся прозрачно для вызывающего кода. Операция загрузки файла на удалённый хост реализована через `asyncssh.start_sftp_client` поверх существующего соединения, что позволяет избежать создания дополнительной SSH-сессии и обеспечивает передачу артефактов в рамках единой уже аутентифицированной сессии.

2.4.7 Восстановление состояния при перезапуске

При старте приложения метод `restore_state` обходит реестр загрузок в Redis и для каждой записи со статусом `RUNNING` выполняет проверку фактического наличия соответствующего процесса на целевом хосте. Если процесс обнаружен, восстанавливается HTTP-клиент прокси-движка с прежними параметрами (адрес хоста и порт). Если процесс не обнаружен — система автоматически инициирует его перезапуск: вызывается внутренний метод `_start_mock_from_data`, который выполняет полный цикл запуска (поиск свободного порта, формирование команды, старт Java-процесса), после чего запись в Redis обновляется новыми значениями `pid`, `port` и `started_at`. Аналогично обрабатывается ситуация, когда поля `pid` или `port` в записи Redis отсутствуют или содержат некорректные значения: система не переводит загрузку в `STOPPED`, а предпринимает попытку повторного запуска. В случае неудачи перезапуска ошибка фиксируется в журнале приложения. Таким образом, механизм восстановления реализует стратегию «наилучшего усилия» (best-effort restart), минимизируя необходимость ручного вмешательства оператора после перезагрузки управляющего сервера.

2.5 Разработка модуля динамического проксирования запросов на базе FastAPI и алгоритма Rate Limiting

2.5.1 Архитектура прокси-модуля

Модуль проксирования реализует функциональность прозрачного обратного прокси-сервера на прикладном уровне: входящий HTTP-запрос, адресованный конкретной заглушке, перенаправляется к Java-процессу этой заглушки, а ответ возвращается инициатору без изменений. Прозрачность означает, что клиент не обязан знать ни реальный адрес, ни порт заглушки – достаточно знать её имя в реестре.

Модуль состоит из двух взаимодействующих программных единиц: реестра HTTP-клиентов `ProxyClientRegistry` и функции непосредственного проксирования `proxy_request`. Маршрутизация входящих запросов обеспечивается FastAPI-роутером `proxy_router`, подключённым к приложению последним – после всех API-маршрутов и маршрутов веб-интерфейса. Это исключает конфликты с системными эндпоинтами при совпадении имён заглушек с зарезервированными путями.

2.5.2 Реестр HTTP-клиентов

Для минимизации задержки при проксировании для каждой заглушки поддерживается отдельный постоянный HTTP-клиент (`httpx.AsyncClient`[21]) с настроенным пулом соединений и поддержкой `keep-alive`. Создание нового TCP-соединения к `upstream` при каждом запросе являлось бы неприемлемым источником задержки в условиях нагрузочного тестирования; повторное использование соединения из пула сводит этот накладные расходы к минимуму.

Класс `ProxyClientRegistry` обеспечивает создание, хранение и корректное закрытие клиентов. Параметры пула соединений фиксированы: до

1000 одновременных соединений, до 200 keep-alive соединений с временем жизни 60 секунд. Реализация реестра приведена в листинге 2.10.

Листинг 2.10 – Реестр HTTP-клиентов ProxyClientRegistry

```
1 class ProxyClientRegistry:
2     """Один httpx.AsyncClient на заглушку с пулом соединений."""
3
4     def __init__(self) -> None:
5         self._clients: dict[str, httpx.AsyncClient] = {}
6         self._lock = asyncio.Lock()
7
8     async def get_or_create(
9         self,
10        mock_name: str,
11        base_url: str,
12        timeout: float | httpx.Timeout,
13    ) -> httpx.AsyncClient:
14        async with self._lock:
15            existing = self._clients.get(mock_name)
16            if existing is not None:
17                return existing
18            limits = httpx.Limits(
19                max_connections=1000,
20                max_keepalive_connections=200,
21                keepalive_expiry=60.0,
22            )
23            t = timeout if isinstance(timeout, httpx.Timeout) else
24                ⇨ httpx.Timeout(timeout)
25            client = httpx.AsyncClient(
26                base_url=base_url.rstrip("/"),
27                timeout=t,
28                limits=limits,
29                follow_redirects=False,
30            )
31            self._clients[mock_name] = client
32            return client
33
34    async def remove(self, mock_name: str) -> None:
35        async with self._lock:
36            client = self._clients.pop(mock_name, None)
37            if client is not None:
38                await client.aclose()78 # корректное закрытие всех
39                ⇨ соединений пула
```

Параметр `follow_redirects=False` установлен намеренно: редиректы, возвращаемые заглушкой, должны быть переданы клиенту в исходном виде, а не обрабатываться прозрачно на уровне прокси. Создание клиента защищено `asyncio.Lock` против состояния гонки при конкурентных запросах к только что зарегистрированной заглушке.

2.5.3 Маршрутизация и обработка запросов

Прокси-роутер регистрирует два catch-all маршрута для всех поддерживаемых HTTP-методов (GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS): `/ {mock_name}` для запросов к корню заглушки и `/ {mock_name} / {path:path}` для запросов с произвольным путём. Оба маршрута делегируют обработку единой внутренней функции `_handle_proxu`, которая реализует полный конвейер обслуживания запроса.

Конвейер обслуживания каждого запроса включает следующие шаги:

а) Чтение пяти полей записи заглушки из Redis одной командой HMGET (hostname, port, status, rate_limit, rate_limit_enabled).

б) Проверка статуса заглушки: если статус отличен от RUNNING – возврат HTTP 503 (Service Unavailable).

в) Чтение глобальных параметров: размера окна Rate Limiter и таймута проксирования из `settings:global`.

г) Проверка Rate Limiter при `rate_limit_enabled = true`: если лимит превышен – инкремент счётчика отклонённых запросов и возврат HTTP 429 (Too Many Requests).

д) Инкремент счётчика успешно проксированных запросов.

е) Получение или создание HTTP-клиента из реестра ProxyClientRegistry.

ж) Чтение тела запроса (для методов, отличных от GET/HEAD/OPTIONS).

з) Выполнение запроса к upstream через `proxy_request` и возврат ответа клиенту.

и) Сохранение метрик конвейера в Redis.

Полная реализация конвейера приведена в листинге 2.11.

Листинг 2.11 – Конвейер обслуживания запроса в `_handle_proxy`

```

1  async def _handle_proxy(
2      mock_name: str, path: str, request: Request,
3      redis: Redis, registry: ProxyClientRegistry,
4      settings: Settings, rate_limiter: RateLimiter,
5  ) -> Response:
6      started_at = time.perf_counter()
7      timeline: dict[str, float] = {}
8      outcome = "ok"
9      upstream_status: int | None = None

10     def _mark(stage: str, t0: float) -> None:
11         timeline[stage] = round((time.perf_counter() - t0) * 1000.0,
12             ↪ 3)
13     try:
14         # Шаг 1: чтение состояния заглушки из Redis (HMGET – 5 полей)
15         t = time.perf_counter()
16         hostname_raw, port_raw, status_raw, rl_raw, rl_on_raw = await
17             ↪ redis.hmget(
18                 f"mocks:{mock_name}",
19                 "hostname", "port", "status", "rate_limit",
20                 ↪ "rate_limit_enabled",
21             )
22         _mark("redis_mock_hmget_ms", t)
23
24         if status_raw is None:
25             raise HTTPException(status_code=404, detail="Заглушка не
26                 ↪ найдена")
27
28         hostname, port, status, rate_limit, rate_limit_enabled = \
29             _parse_mock_record({...})    # разбор строк в
30             ↪ типизированные поля
31
32         if status != MockStatus.RUNNING:
33             raise HTTPException(status_code=503, detail="Заглушка не
34                 ↪ запущена")
35
36         # Шаг 3: глобальные параметры (таймаут, размер окна)
37         window_size, timeout_sec = await _load_proxy_runtime(redis,
38             ↪ settings)

```

Все промежуточные этапы конвейера фиксируются в словаре `timeline` с точностью до 0.001 мс посредством вспомогательной функции `_mark`. Это позволяет на уровне Prometheus-метрик разделить задержку Redis, задержку Rate Limiter и задержку upstream, что критично для диагностики узких мест при нагрузочном тестировании.

2.5.4 Функция прозрачного проксирования

Функция `proxy_request` выполняет непосредственную передачу запроса к upstream. Прозрачность достигается следующими механизмами:

- HTTP-метод, тело запроса и `query string` передаются к upstream без изменений.
- Заголовки запроса пересылаются целиком, за исключением заголовков `Host` и `Content-Length` – их значения должны быть сформированы заново для нового соединения, а не скопированы из исходного запроса.
- Код статуса, заголовки и тело ответа upstream возвращаются клиенту без каких-либо модификаций.

Обработка ошибок транспортного уровня унифицирована: исключение `httpx.TimeoutException` транслируется в HTTP 504 (Gateway Timeout), любое иное исключение `httpx.RequestError` – в HTTP 502 (Bad Gateway). Это обеспечивает предсказуемое поведение для клиентов нагрузочного тестирования при недоступности заглушки.

Реализация фильтрации заголовков и сборки пути запроса приведена в листинге 2.12.

Листинг 2.12 – Фильтрация заголовков и сборка пути запроса

```
1 # Заголовки, которые не пересылаются к upstream
2 _STRIP_REQUEST_HEADERS = frozenset({"host", "content-length"})

3 def _filtered_request_headers(headers) -> dict[str, str]:
4     """Копирует заголовки запроса, исключая host и content-length."""
5     return {
6         key: value
7         for key, value in headers.items()
8         if key.lower() not in _STRIP_REQUEST_HEADERS
9     }

10 def _request_path(path: str, query_string: str) -> str:
11     """Собирает целевой URL-путь с query string."""
12     url_path = "/" if not path else "/" + path.lstrip("/")
13     return f"{url_path}?{query_string}" if query_string else url_path
```

Помимо основного результата, функция `proxy_request` возвращает объект `ProxyRequestObservation`, содержащий детальную разбивку задержек по этапам HTTP-взаимодействия: время ожидания соединения из пула, время установки соединения, время передачи и время чтения ответа. Эти данные собираются через механизм `httpx transport events` и впоследствии экспортируются как метрики `Prometheus`.

2.5.5 Алгоритм Rate Limiting: Fixed Window Counter

Алгоритм ограничения интенсивности запросов реализован в классе `RateLimiter` по паттерну «Счётчик с фиксированным временным окном» (`Fixed Window Counter`). Алгоритм выбран как наиболее эффективный с точки зрения операционной стоимости: для проверки лимита требуется не более двух атомарных операций `Redis` (`INCR` и `EXPIRE`), что обеспечивает субмиллисекундное время выполнения даже под пиковой нагрузкой.

Структура ключа счётчика: `rate:{filename}:{window}`, где метка окна вычисляется как целочисленное деление текущего `Unix-timestamp` на

размер окна в секундах. Все запросы, поступившие в пределах одного временного окна, инкрементируют единый счётчик. Реализация класса приведена в листинге 2.13.

Листинг 2.13 – Реализация алгоритма Fixed Window Counter

```
1 class RateLimiter:
2     """Ограничение частоты запросов на заглушку в фиксированных
   ↪ временных окнах."""
3
4     def __init__(self, redis: Redis) -> None:
5         self._redis = redis
6
7     async def check(self, filename: str, limit: int,
8                     window_size: int = 1) -> bool:
9         """Инкрементирует счётчик текущего окна.
10          Возвращает True, если запрос в пределах лимита."""
11         window = int(time.time()) // window_size
12         key = f"rate:{filename}:{window}"
13
14         # Атомарный инкремент: Redis гарантирует неделимость операции
15         count = await self._redis.incr(key)
16
17         # TTL устанавливается только при создании нового счётчика
   ↪ (count == 1)
18         if count == 1:
19             await self._redis.expire(key, window_size * 2)
20
21         return count <= limit # True -> разрешить, False -> HTTP 429
```

Атомарность операции INCR[25] является принципиальным свойством данного подхода: Redis выполняет инкремент как единую неделимую операцию, что гарантирует корректный подсчёт при конкурентном поступлении тысяч запросов без использования транзакций или блокировок на стороне приложения.

TTL ключа устанавливается равным удвоенному размеру окна исключительно при создании счётчика, то есть при первом запросе в новом времен-

ном окне (когда INCR возвращает 1). Такой подход обеспечивает автоматическую очистку устаревших счётчиков без накопления «мёртвых» ключей в памяти Redis.

2.5.6 Управление Rate Limiter через API

Включение и отключение Rate Limiter для конкретной заглушки производится в режиме реального времени через PATCH-эндпоинт без остановки и перезапуска заглушки. Операция обновляет единственное поле `rate_limit_enabled` в Hash-записи заглушки в Redis; начиная со следующего входящего запроса, конвейер `_handle_proxy` считывает обновлённое значение и применяет его. Описание эндпоинта приведено в таблице 8.

Таблица 8 – Эндпоинты управления Rate Limiter

Метод	Путь	Описание
PATCH	<code>/api/v1/mocks/{filename}/rate-limit</code>	Включить или отключить Rate Limiter (тело: <code>{"enabled": true/false}</code>)
PATCH	<code>/api/v1/mocks/{filename}/config</code>	Обновить порог лимита (<code>rate_limit</code>) и/или JVM-аргументы

Порог лимита (`rate_limit`) – максимальное число запросов в одном временном окне – можно изменить отдельным запросом к эндпоинту `/api/v1/mocks/{filename}/config`, не перезапуская заглушку. Новое значение вступает в силу в момент создания следующего временного окна:

текущее окно уже имеет накопленный счётчик, и смена порога повлияет на него только если новый лимит превышает уже накопленный счётчик.

2.5.7 Известные ограничения алгоритма и их приемлемость

Алгоритм Fixed Window Counter имеет известное теоретическое ограничение: на границе двух временных окон возможен кратковременный всплеск нагрузки до $2 \times \text{limit}$ запросов – по **limit** в конце первого окна и по **limit** в начале второго. Более точными алгоритмами являются Sliding Window Log и Sliding Window Counter, однако они требуют хранения временных меток каждого запроса (Sliding Window Log) или двух счётчиков со взвешенным усреднением (Sliding Window Counter), что увеличивает нагрузку на Redis пропорционально интенсивности трафика.

В контексте нагрузочного тестирования задача Rate Limiter состоит в защите заглушки от перегрузки, а не в точном соблюдении квоты с субсекундной точностью. Кратковременный всплеск на границе окна не создаёт рисков для корректности тестового сценария: статистические показатели нагрузочного теста – пропускная способность, время отклика и процент ошибок – рассчитываются на интервалах в несколько секунд, что многократно превышает размер одного временного окна. Таким образом, Fixed Window Counter является оптимальным выбором с точки зрения соотношения точности и операционной стоимости для данного класса задач.

2.6 Проектирование и программная реализация пользовательского интерфейса (Dashboard, управление кластером).

2.7 Реализация механизма экспорта метрик мониторинга

2.8 Выводы по второй главе.

3 ВНЕДРЕНИЕ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ РАЗРАБОТАННОГО ПРОГРАММНОГО КОМПЛЕКСА

3.1 Описание процесса развертывания системы в среде ОС Astra Linux (настройка Uvicorn/Gunicorn и службы Redis).

3.2 Тестирование механизмов автоматического восстановления состояний (персистентность Redis AOF) и перезапуска сервисов.

3.3 Экспериментальная оценка производительности прокси-модуля и эффективности алгоритма Rate Limiting под нагрузкой.

3.4 Анализ работы системы в кластерном режиме и оценка алгоритмов распределения нагрузки между узлами.

3.5 Рекомендации по дальнейшему развитию системы (шаблонизация конфигураций, Self-healing).

3.6 Выводы по третьей главе.

ЗАКЛЮЧЕНИЕ

Описание проделанной работы

В результате работы были выполнены следующие задачи:

1. Задача 1
2. Задача 2

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. WildFly — JBoss Application Server Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.wildfly.org/documentation/>.
2. Eclipse GlassFish Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://glassfish.org/documentation>.
3. asyncio — Asynchronous I/O. Python 3.12 Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://docs.python.org/3/library/asyncio.html>.
4. FastAPI Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://fastapi.tiangolo.com/>.
5. ASGI (Asynchronous Server Gateway Interface) Specification. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://asgi.readthedocs.io/en/latest/>.
6. Pydantic Documentation — Data validation using Python type hints. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://docs.pydantic.dev/>.
7. Uvicorn — An ASGI web server, for Python. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.uvicorn.org/>.
8. Redis Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://redis.io/docs/>.
9. Redis Persistence: Append Only File. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://redis.io/docs/management/persistence/>.
10. Astra Linux Special Edition — официальная документация. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://wiki.astralinux.ru/>.
11. WireMock Documentation. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <https://wiremock.org/docs/>.
12. MockServer Documentation. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <https://www.mock-server.com/>.

13. Mountebank Documentation. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <http://www.mbtest.org/docs/>.
14. Apache Tomcat 9 Documentation. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <https://tomcat.apache.org/tomcat-9.0-doc/index.html>.
15. Tomcat Clustering/Session Replication How-To. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <https://tomcat.apache.org/tomcat-9.0-doc/cluster-howto.html>.
16. Apache Tomcat Configuration Reference: The Rate Limit Filter. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. https://tomcat.apache.org/tomcat-9.0-doc/config/filter.html#Rate_Limit_Filter.
17. OpenJDK 17 Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://openjdk.org/projects/jdk/17/>.
18. The C4 model for visualising software architecture. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://c4model.com/>.
19. Jinja2 Documentation — The Python Template Engine. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://jinja.palletsprojects.com/>.
20. Paramiko Documentation — SSH2 protocol library for Python. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.paramiko.org/>.
21. HTTPX — A next-generation HTTP client for Python. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.python-httpx.org/>.
22. Cryptography Library — Fernet (symmetric encryption). – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://cryptography.io/en/latest/fernet/>.
23. Prometheus Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://prometheus.io/docs/introduction/overview/>.

24. Prometheus Exposition Formats. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. https://prometheus.io/docs/instrumenting/exposition_formats/.

25. Redis Commands: INCR — Increment the integer value of a key. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://redis.io/commands/incr/>.

26. Web Content Accessibility Guidelines (WCAG) 2.1. – 2018. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.w3.org/TR/WCAG21/>.

ПРИЛОЖЕНИЕ А